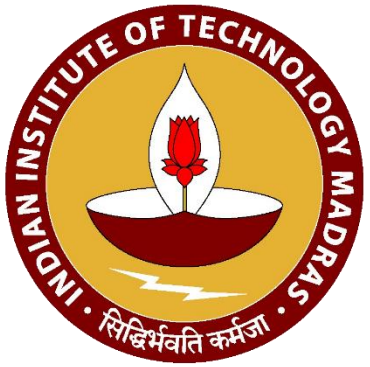
DA5400: Foundations of Machine Learning

Topic 6: Neural Networks

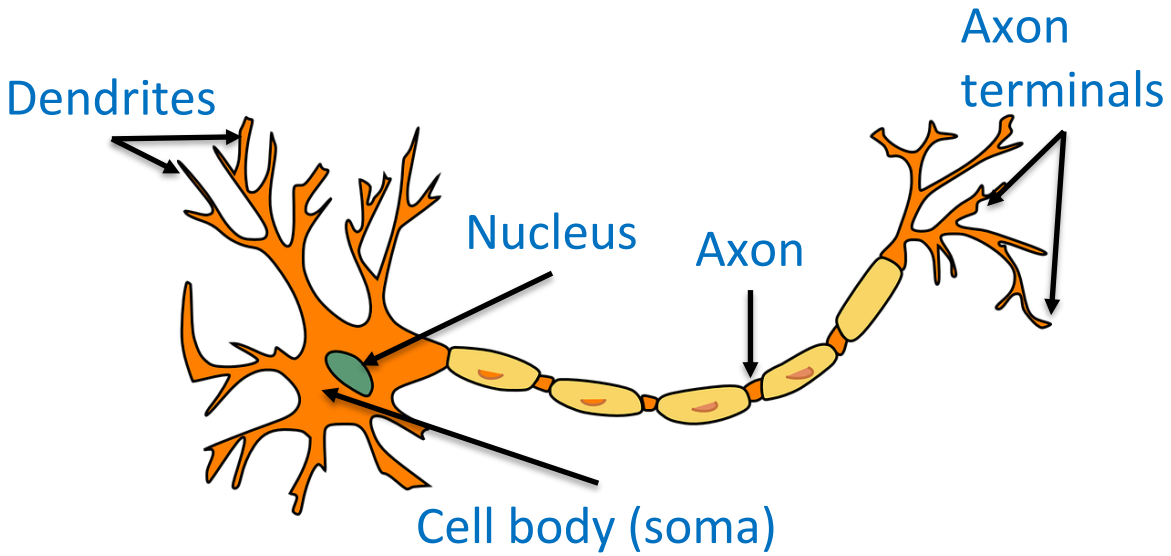


Contents

- Motivation: Biological Neuron to Artificial Neuron
- Perceptron Model of Artificial Neuron
- Artificial Neural Network (ANN or DNN)
- Activation Function and Types
- Universal Approximation Theorem
- Supervised Learning using DNN
- Batch Normalisation and Regularisation
- Hyperparameter Tuning

Biological Neuron

- Basic working unit of the brain and nervous system
- Close to 100 billion interconnected neurons in a human brain
- Function together to aid decision making
- Parts and Functioning:
  - Dendrite: Takes signals (stimulus) from the other neurons or other cells in the body
  - Cell body (soma): Processes the signal and may or may not fire the neuron – excitation and inhibition
  - Axon: Transmits the output (response) to other neurons or cells

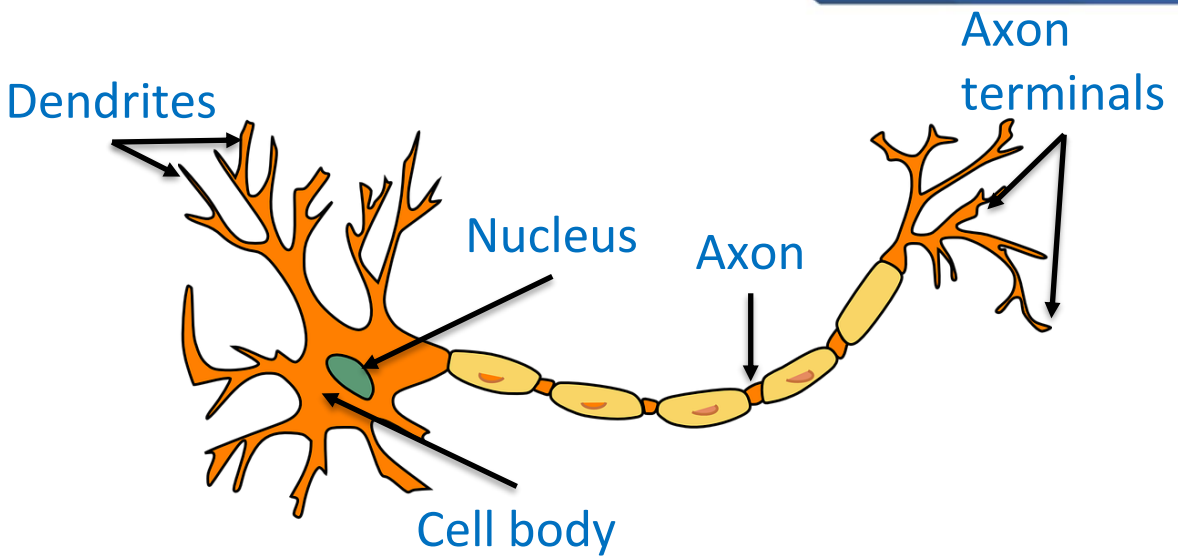


Biological Neuron

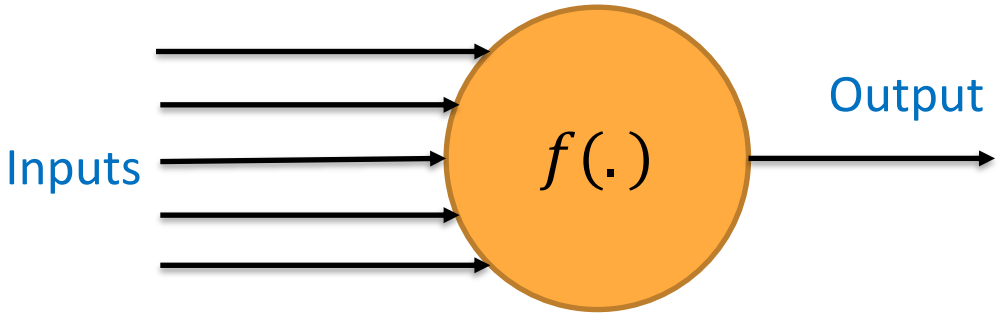
Biological Neuron to Artificial Neuron

Artificial neuron

- Mathematical model of a biological neuron
- Mimics the functioning of a biological neuron
- Takes input in the form of numbers
- Processes the input to give out an output
- Output = f(inputs)
- Different models of artificial neurons have been developed based on this idea



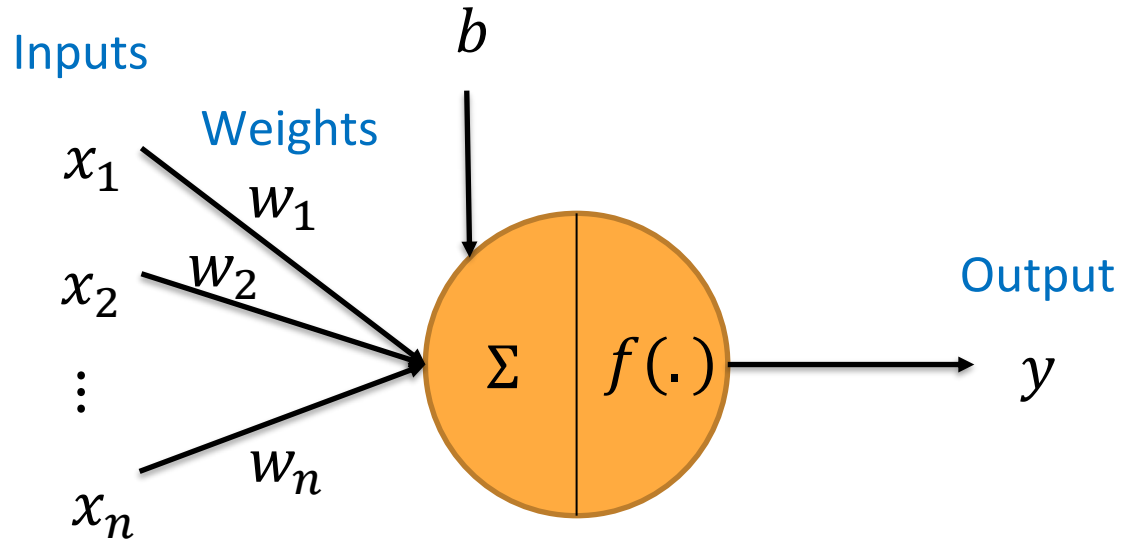
Biological Neuron



Artificial Neuron

Artificial Neuron: Perceptron Model

- Inputs $(x_1, x_2, \ldots, x_n)$ are real numbers
- Neuron takes the weighted combination of inputs
- Bias (b) is added to weighted inputs
- Bias captures factors affecting the output not captured through inputs
- Weighted input passes through an activation function to give output (y)



Perceptron Model

$$a = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

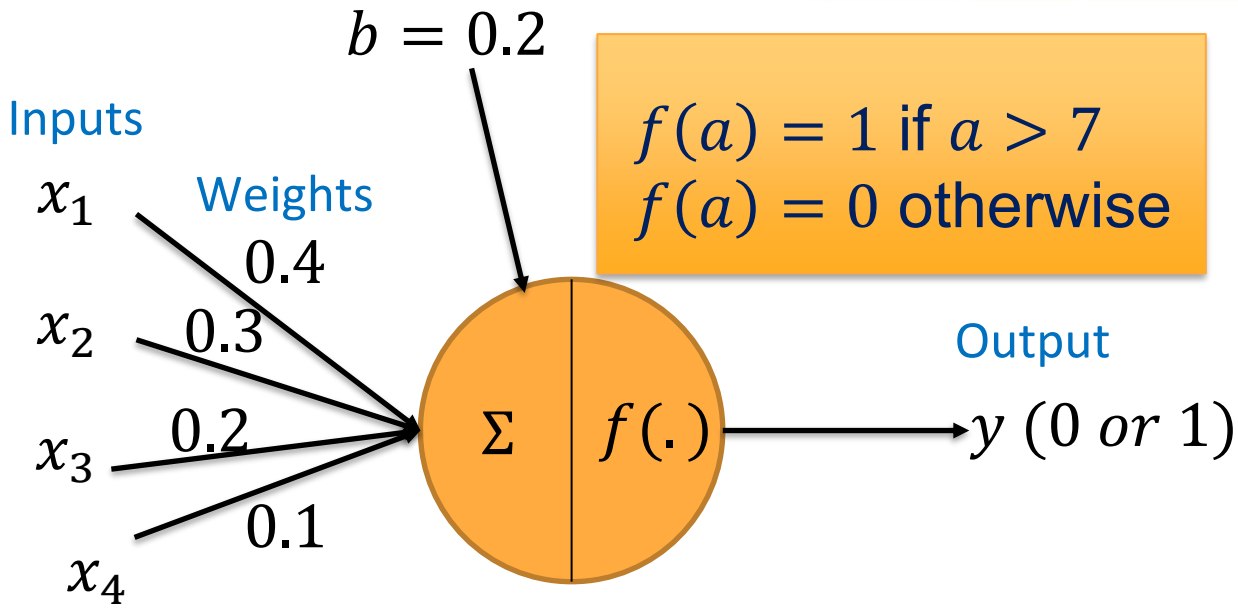
$$y = f(a) = f(w_1x_1 + w_2x_2 + \cdots + w_nx_n + \mathrm{b})$$

Artificial Neuron: A Simple Example

- Using an artificial neuron to decide whether to watch a movie or not

Feature	Value for Movie 1	Value for Movie 2
Lead Actor (x_1)	10	7
Director (x_2)	8	5
Trill factor (x_3)	8	9
Run time (x_4)	9	5

- Bias could be the general interest towards watching a movie



$f(a) = 1$ if $a > 7$
$f(a) = 0$ otherwise

Perceptron Model

Movie 1

$a = 9.1$

$y = f(a) = 1$

Movie 2

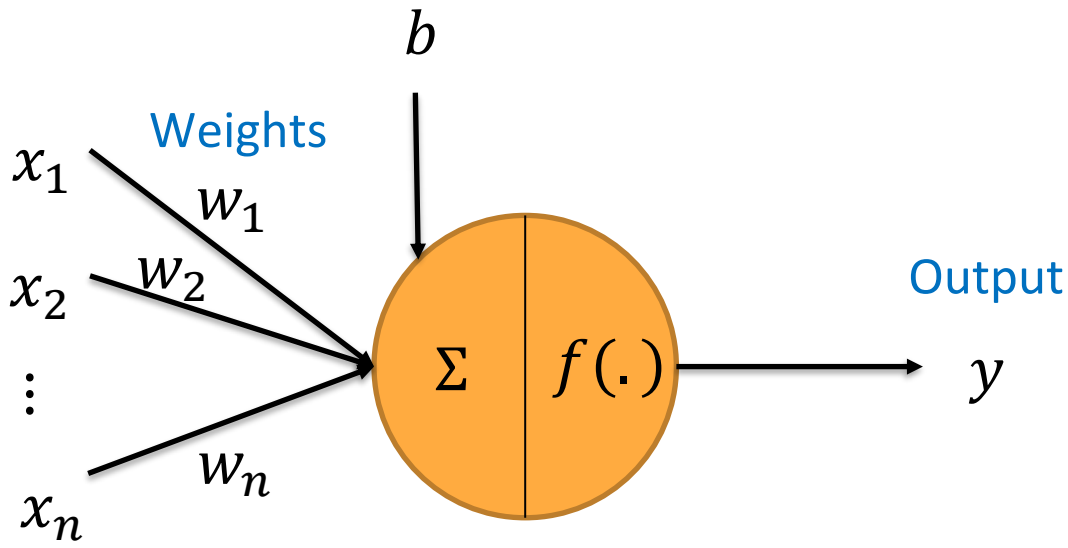
$a = 6.9$

$y = f(a) = 0$

Artificial Neural Network

Artificial Neural Network: Motivation

- One neuron is not sufficient to take complex decisions (complex functions)
- Again, inspired by brain neural network, artificial neural network was developed
- In the brain, many neurons are involved in taking a decision
- All the neurons are inter-connected in the brain
- They are arranged hierarchically in layers



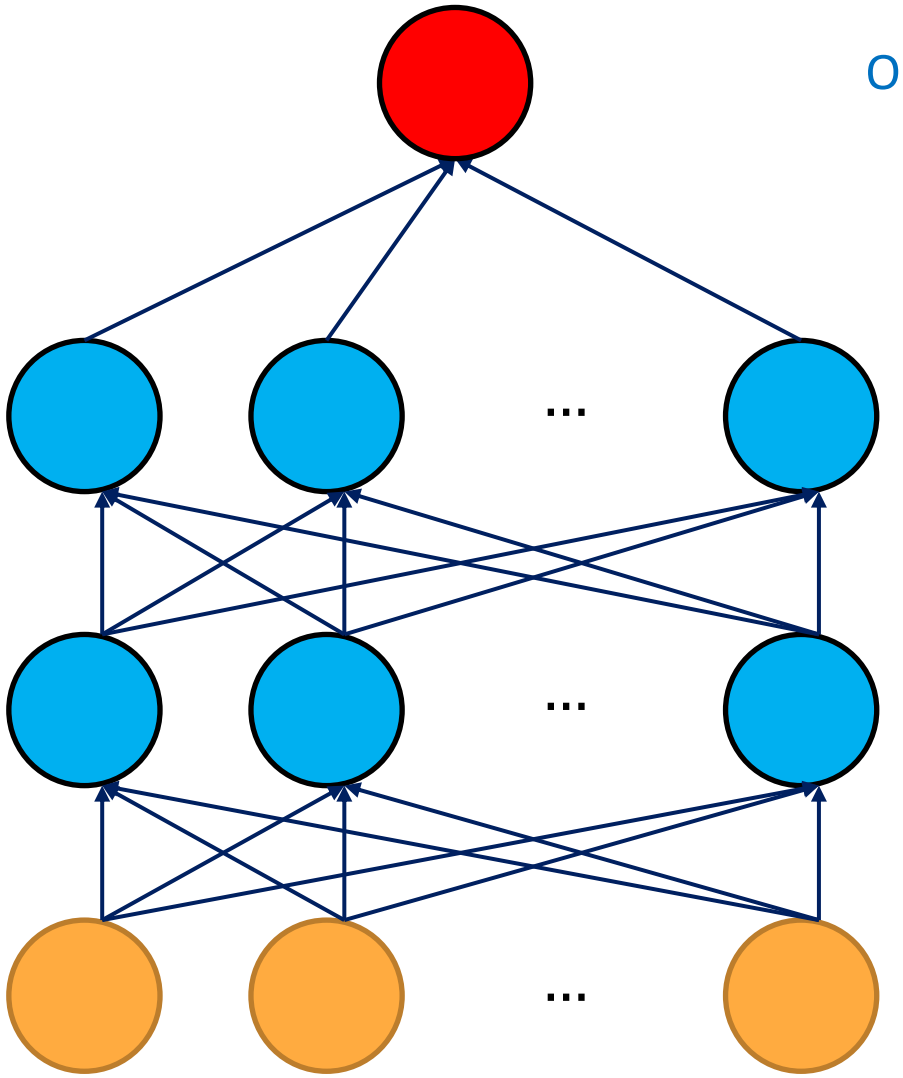
Perceptron Model

$$a = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

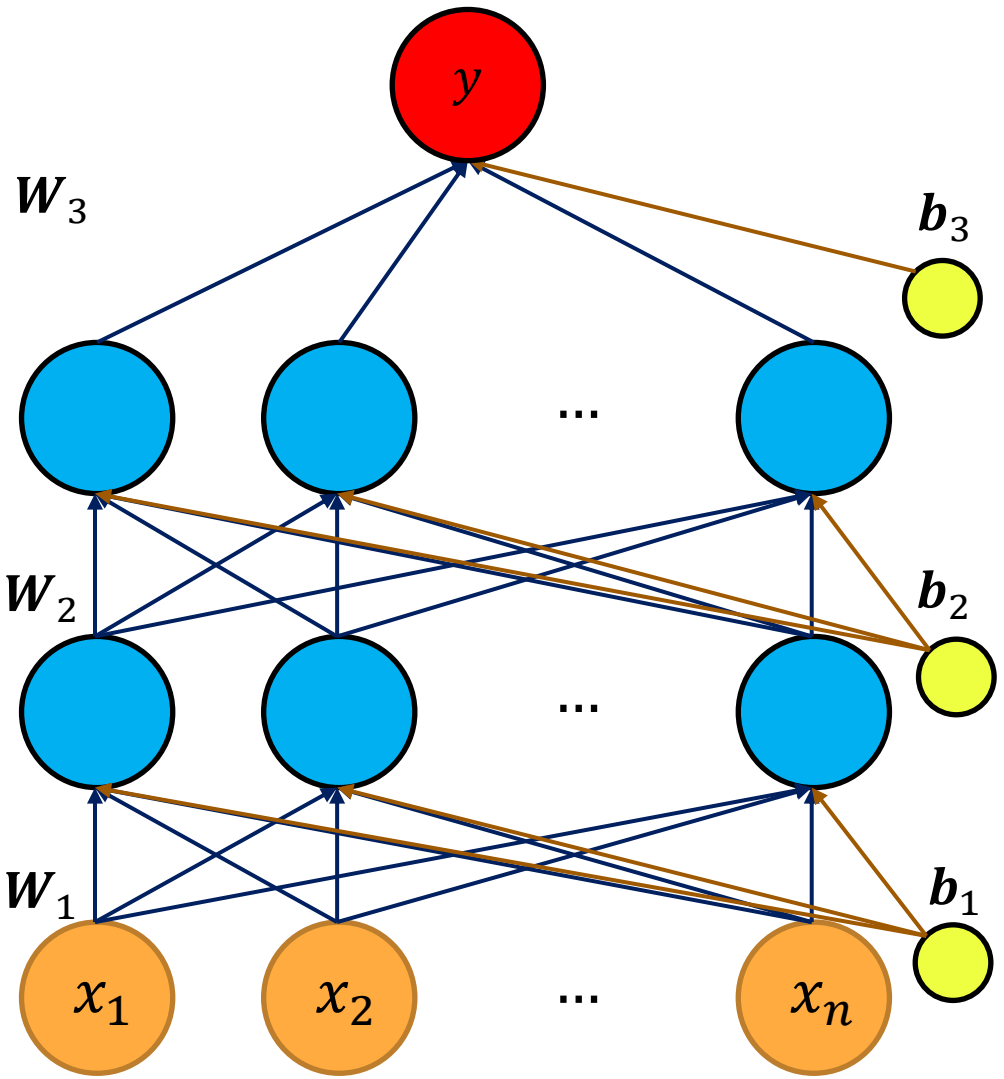
$$y = f(a) = f(w_1x_1 + w_2x_2 + \cdots + w_nx_n + \mathrm{b})$$

Artificial Neural Network (ANN)

- ANN consists of multiple layers with multiple neurons in each layer (hidden layers)
- Each neuron in hidden and output layers represents a perceptron model
- Every neuron in one layer is connected to every neuron in the successive layer
- Output of one neuron is passed as input to every neuron in the next layer



ANN Architecture



- Input layer (0^{th}) with n inputs

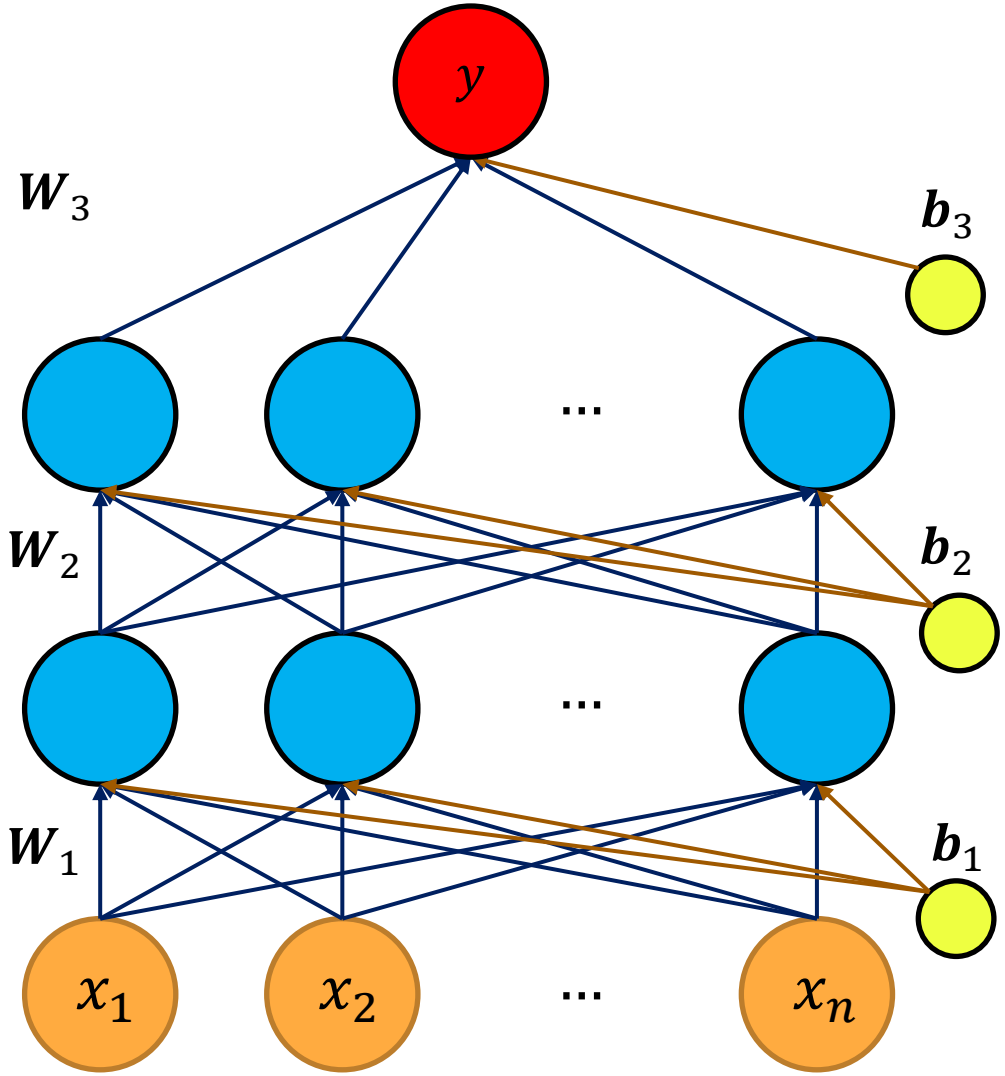
$$\boldsymbol{x} = [x_1 \quad x_2 \quad \dots \quad x_n]^T$$

- $L-1$ hidden layers with m neurons each
- Output layer (L^{th}) with k neurons
- $\boldsymbol{W}_i$ is the matrix containing the weights between layers $i-1$ and i $(0 < i \leq L)$
- $\boldsymbol{b}_i$ is the vector representing the biases

$$\boldsymbol{W}_1 = \begin{bmatrix} w_{11}^1 & \dots & w_{1n}^1 \\ \vdots & \ddots & \vdots \\ w_{m1}^1 & \dots & w_{mn}^1 \end{bmatrix}_{m \times n} \qquad \boldsymbol{b}_1 = \begin{bmatrix} b_1^1 \\ \vdots \\ b_m^1 \end{bmatrix}$$

$$\boldsymbol{W}_i = \begin{bmatrix} w_{11}^i & \dots & w_{1m}^i \\ \vdots & \ddots & \vdots \\ w_{m1}^i & \dots & w_{mm}^i \end{bmatrix}_{m \times m} \qquad \boldsymbol{b}_\mathrm{i} = \begin{bmatrix} b_1^i \\ \vdots \\ b_m^i \end{bmatrix}$$

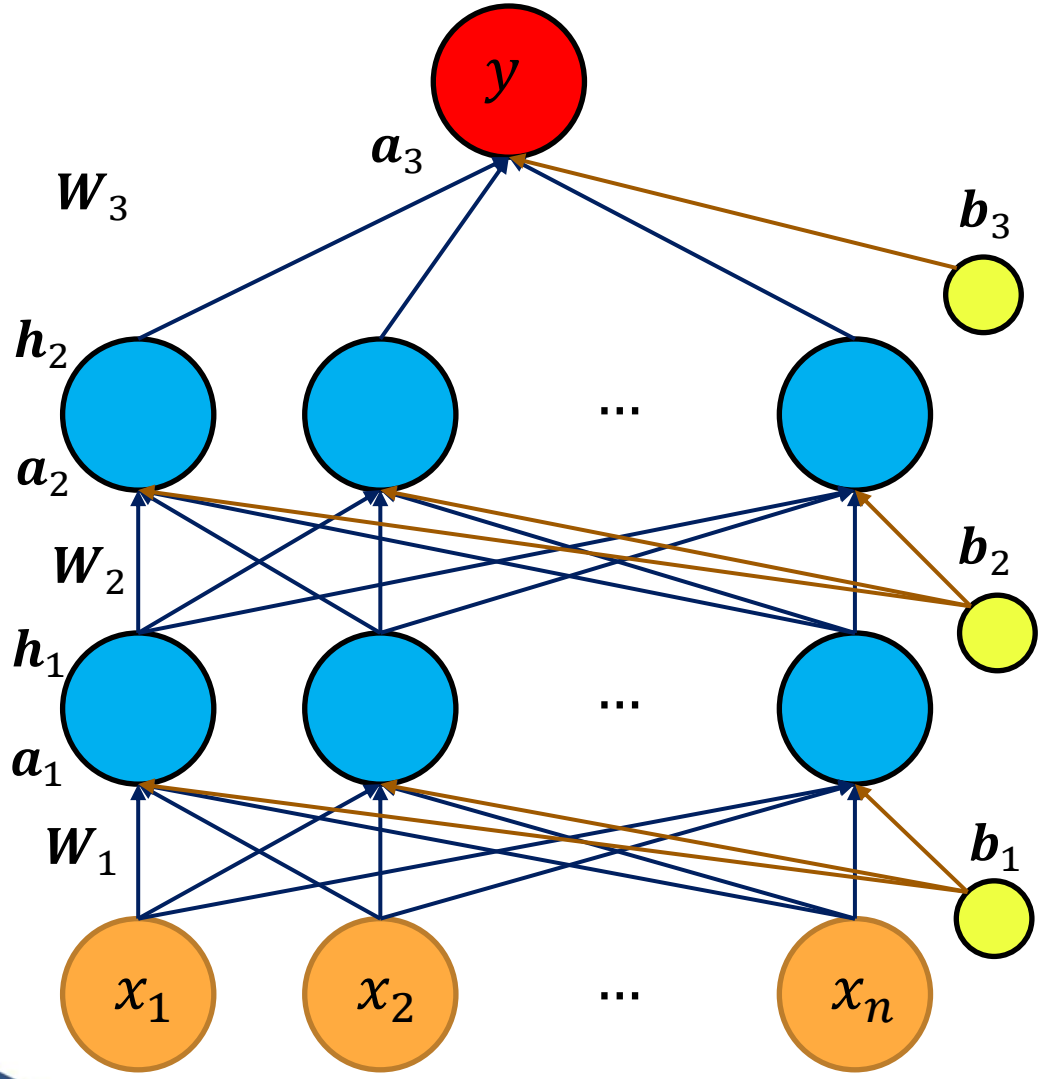
ANN Architecture



$$\boldsymbol{W}_L = \begin{bmatrix} w_{11}^L & \cdots & w_{1m}^L \\ \vdots & \ddots & \vdots \\ w_{k1}^L & \cdots & w_{km}^L \end{bmatrix}_{k \times m} \qquad \boldsymbol{b}_{\mathrm{L}} = \begin{bmatrix} b_1^L \\ \vdots \\ b_k^L \end{bmatrix}$$

- For a single output, $\boldsymbol{W}_L$ will be a vector and $\boldsymbol{b}_L$ will be a scalar
- Each neuron in hidden and output layers has an activation function
- If there are more than 3 hidden layers, then ANN is referred to as Deep Neural Network (DNN) – Depth refers to the number of layers

DNN Feed Forward Calculation



- Feed forward calculation involves finding the output as a function of input, weights and biases
- Input to activation function at layer 0:

$$\boldsymbol{a}_1 = \boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1$$

- Activation at hidden layer 1:

$$\boldsymbol{h}_1 = g_h(\boldsymbol{a}_1)$$

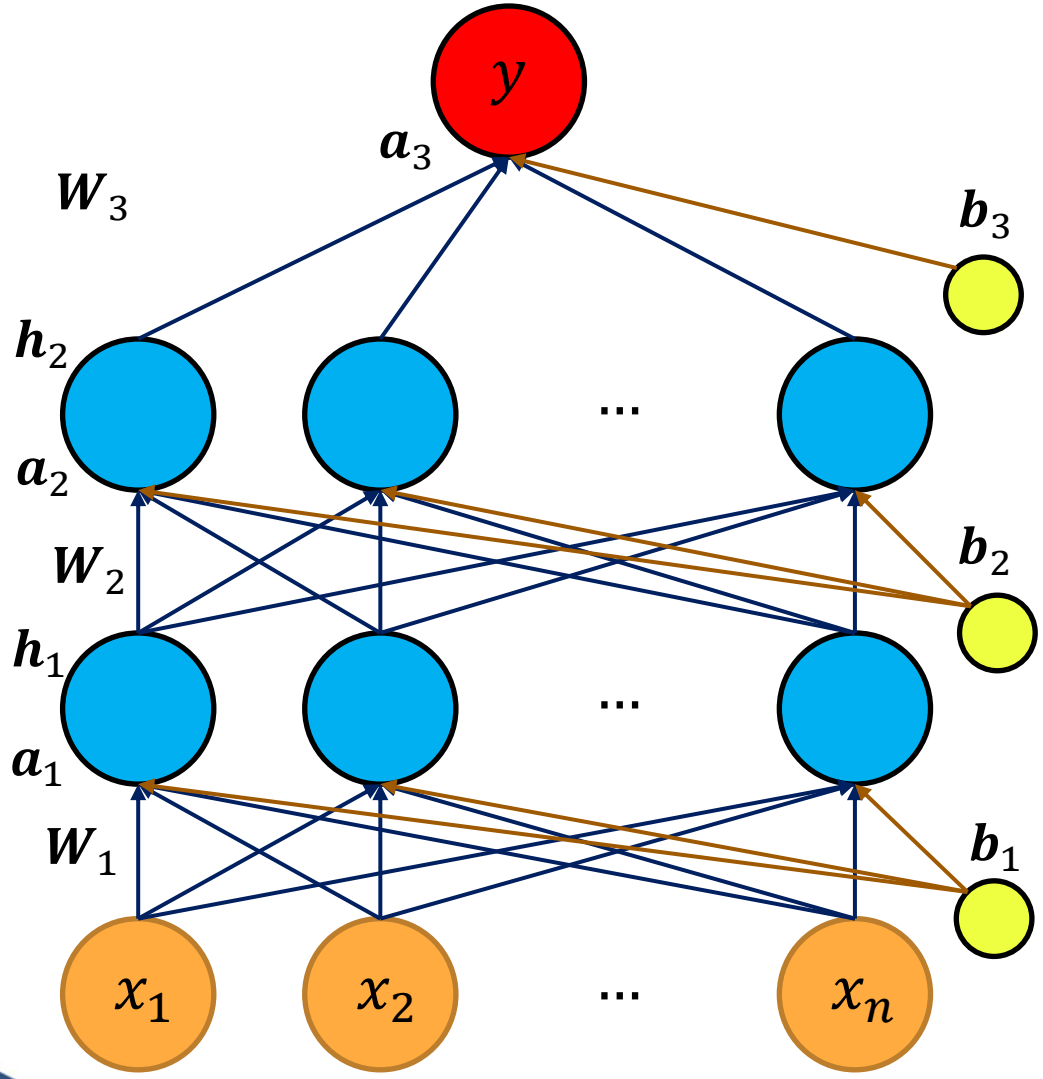
- $\boldsymbol{h}_i$ is output vector at layer i
- g_h is the activation function which maps vector $\boldsymbol{a}_i$ to vector $\boldsymbol{h}_i$
- Input to activation function at hidden layer i:

$$\boldsymbol{a}_i = \boldsymbol{W}_i\boldsymbol{h}_{i-1} + \boldsymbol{b}_i$$

- Activation at hidden layer i:

$$\boldsymbol{h}_i = g_h(\boldsymbol{a}_i) = g_h(\boldsymbol{W}_i\boldsymbol{h}_{i-1} + \boldsymbol{b}_i)$$

DNN Feed Forward Calculation



$$\boldsymbol{a}_1 = \boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1$$
$$\boldsymbol{h}_1 = g_h(\boldsymbol{a}_1)$$
$$\boldsymbol{a}_i = \boldsymbol{W}_i\boldsymbol{h}_{i-1} + \boldsymbol{b}_i$$
$$\boldsymbol{h}_i = g_h(\boldsymbol{a}_i) = g_h(\boldsymbol{W}_i\boldsymbol{h}_{i-1} + \boldsymbol{b}_i)$$

- Input to activation function at output layer L:

$$\boldsymbol{a}_L = \boldsymbol{W}_L\boldsymbol{h}_{L-1} + \boldsymbol{b}_L$$

- Activation at output layer:

$$y = g_o(\boldsymbol{a}_L) = g_o(\boldsymbol{W}_L\boldsymbol{h}_{L-1} + \boldsymbol{b}_L)$$

- Model (function) being approximated by the DNN (assuming L=3):

$$\boldsymbol{y} = g_o(\boldsymbol{W}_3(\boldsymbol{W}_2(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2) + \boldsymbol{b}_3)$$
$$\boldsymbol{y} = f(\boldsymbol{x})$$

Types of Activation Functions

Activation Function

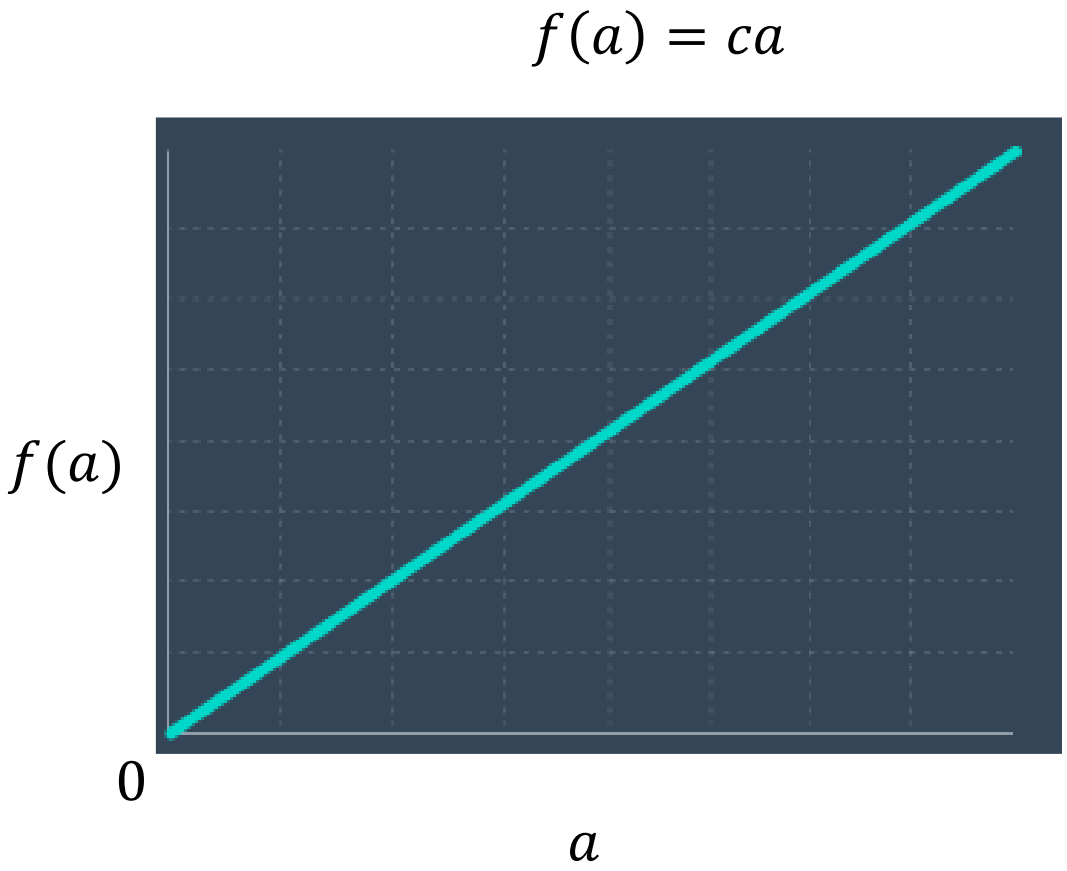
- Activation function is like a gate between the input and output of a neuron
- Purpose: To introduce non-linearity into the function and enable a neural network represent complex functions
- It affects the overall DNN output
- Types of activation function:
  1. Linear activation function
  2. Sigmoid activation function
  3. Softmax activation function
  4. Tanh activation function
  5. ReLU activation function

Linear Activation Function

- Output is directly proportional to the input

$$f(a) = ca$$

- Output can take any real number
- Generally used in the output layer of regression problem



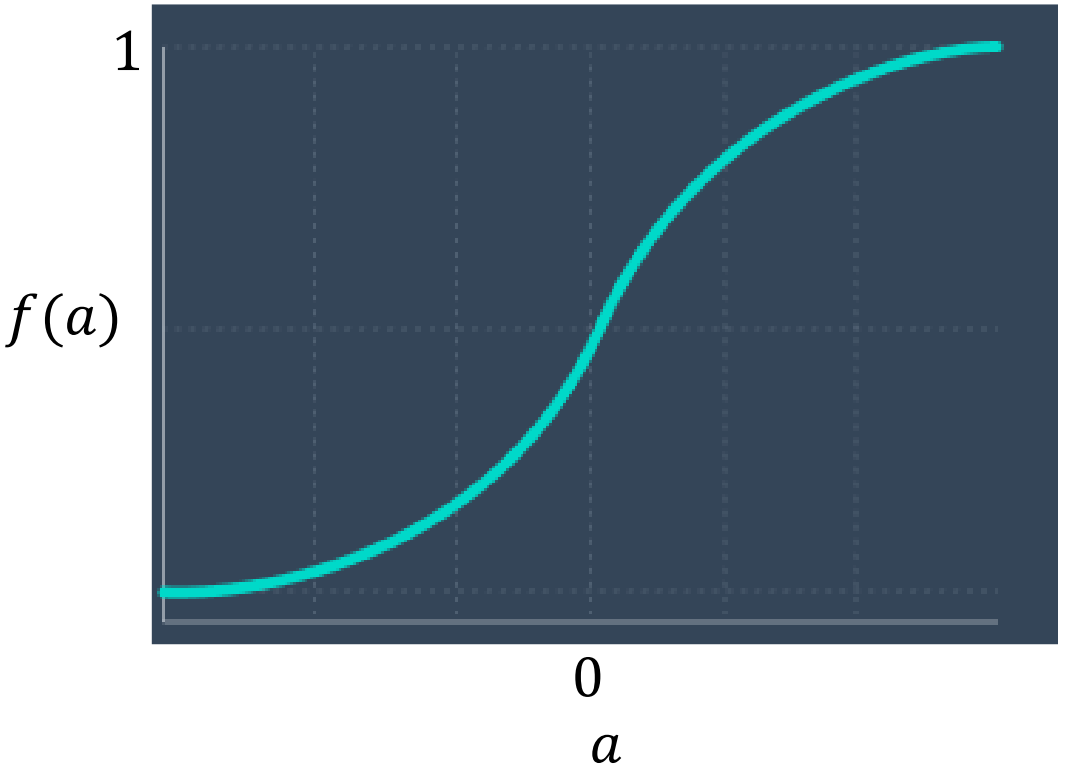
Sigmoid Activation Function

- Any value of input is mapped to a value between 0 and 1

$$f(a) = \frac{1}{1 + e^{-a}}$$

- Useful when the expected output is a probabilistic value between 0 and 1

$$f(a) = \frac{1}{1 + e^{-a}}$$



Softmax Activation Function

- Sigmoid function gives a value between 0 and 1, and can be used for binary classification
- However, sigmoid cannot be used to output multiple probability values which add upto 1 (multi-class)
- Softmax function is an extension of sigmoid function
- Softmax calculates the relative probabilities of multiples classes and ensures that total probability is 1

Input to output layer of a DNN

$$\boldsymbol{a} = [a_1 \quad a_2 \quad \dots \quad a_k]$$

Total Probability = 1

Binary Classification

Total Probability = 1

Multi-class Classification

$$f(a_i) = \frac{e^{a_i}}{\sum_{j=1}^{k} e^{a_j}}$$

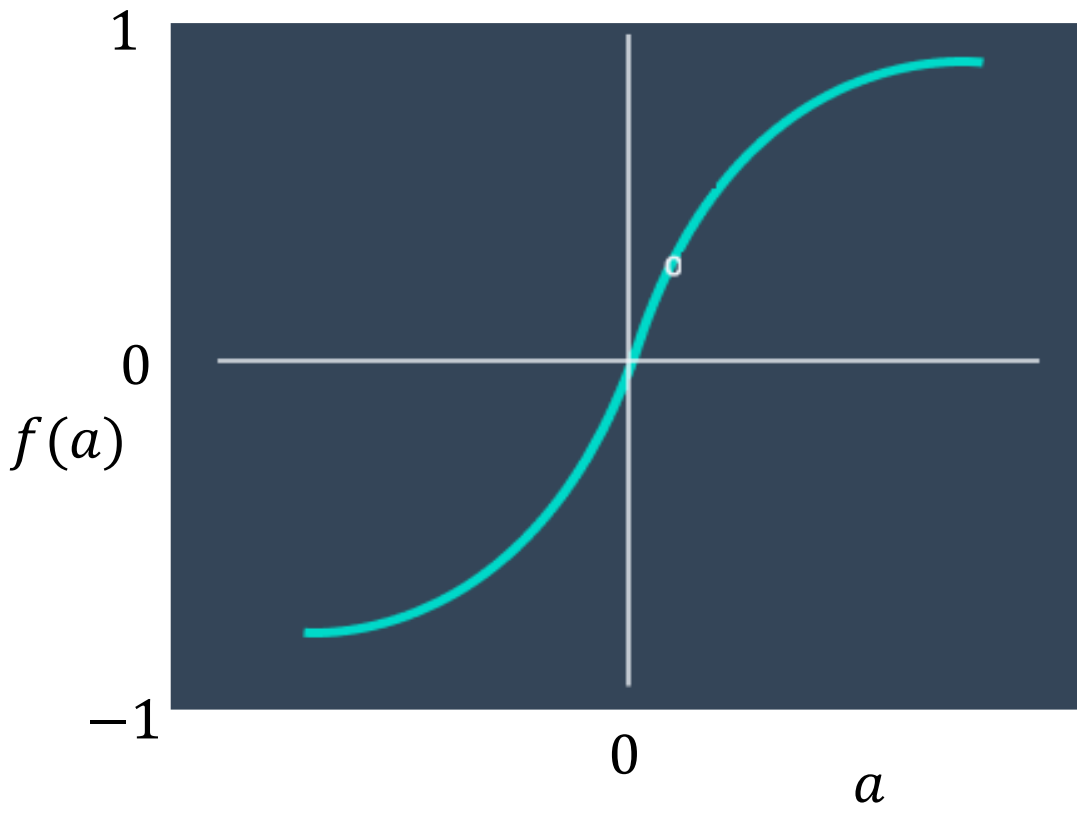
Tanh Activation Function

- Any value of input is mapped to a value between -1 and 1

$$f(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}$$

- Positive values between 0 and 1
- Negative values between -1 and 0
- Output is zero centered can help optimization compared with sigmoid in some settings

$$f(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}$$



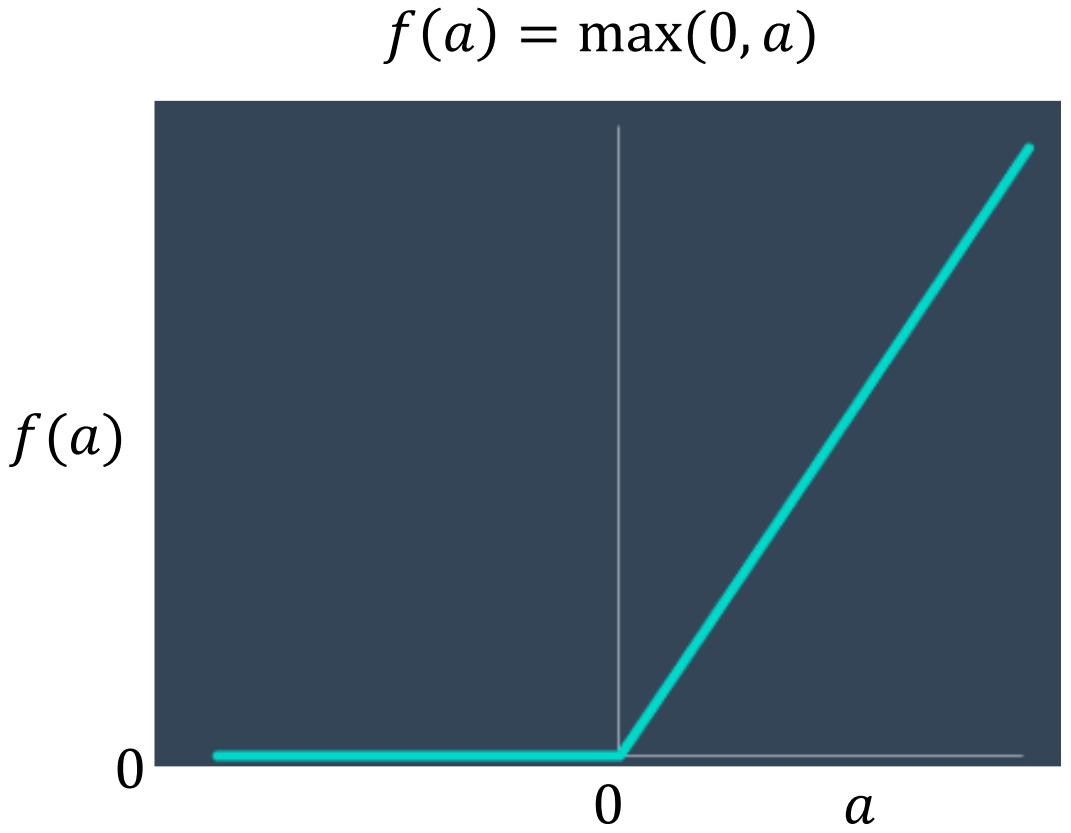
ReLU Activation Function

Rectified Linear Unit

- All positive values go through directly while all negative values are mapped to zero

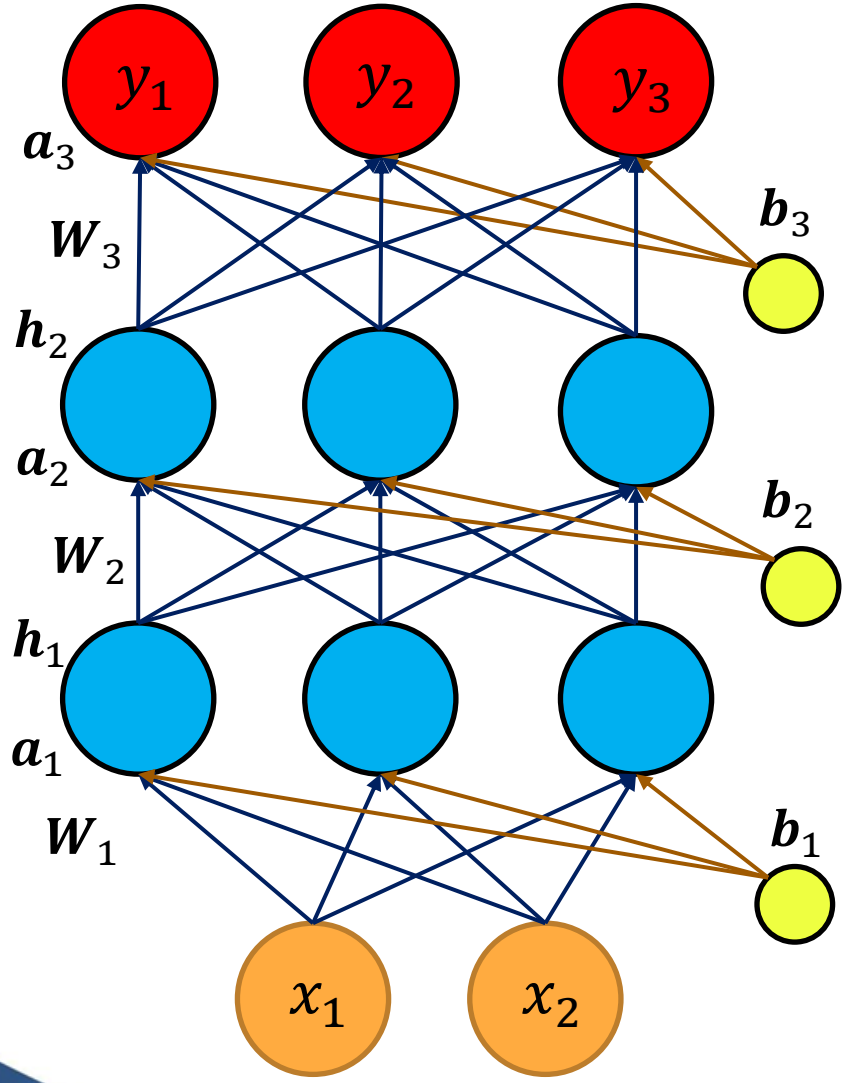
$$f(a) = \max(0, a)$$

- ReLU is one of the most popular activation functions and has many variants



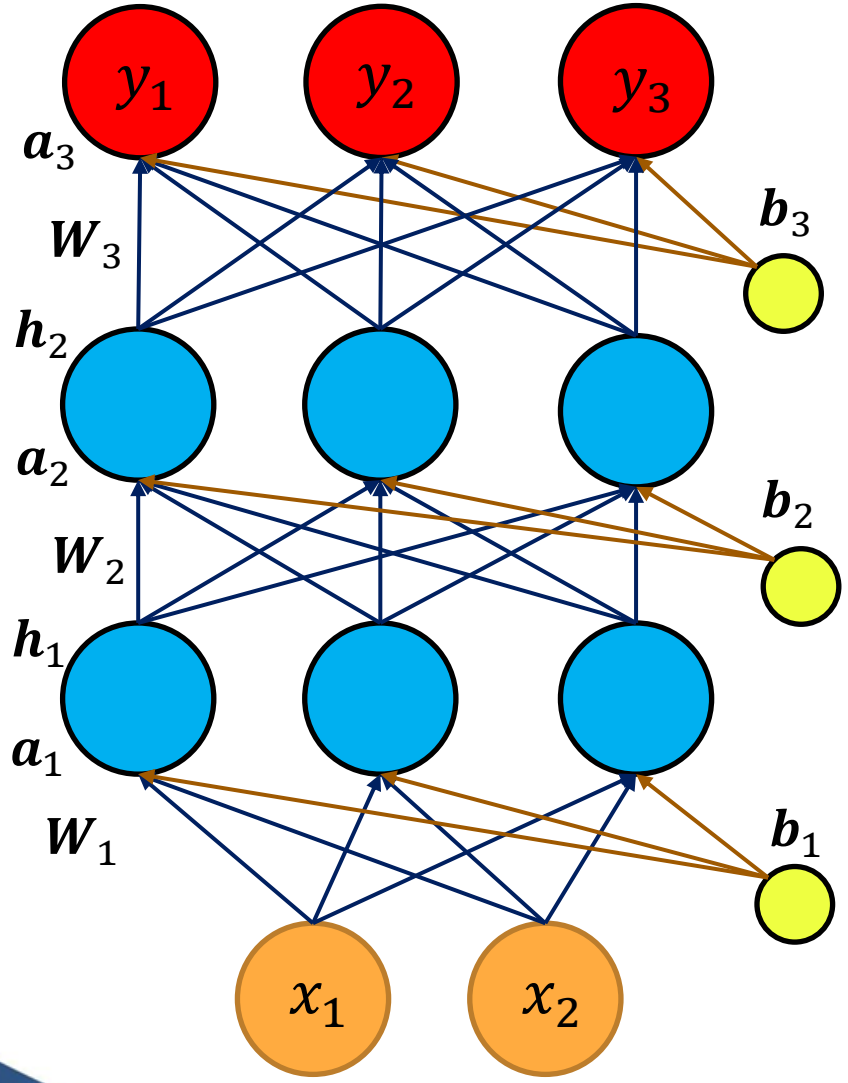
Feed Forward Calculation - Example

Feed Forward Calculation - Example



- Task: To classify a person as underweight (Class 1), normal weight (Class 2) or overweight (Class 3) given the height (x_1) and weight as input features (x_2)
- NN Architecture: A neural network with 2 inputs, 2 hidden layers with 3 neurons each and 3 outputs
- 3 outputs to predict probability of 3 classes i.e., y_1 is probability that person is underweight (class 1) and so on
- Activation functions: All hidden layer neurons have ReLU activation and output layer neurons have softmax activation

Feed Forward Calculation - Example



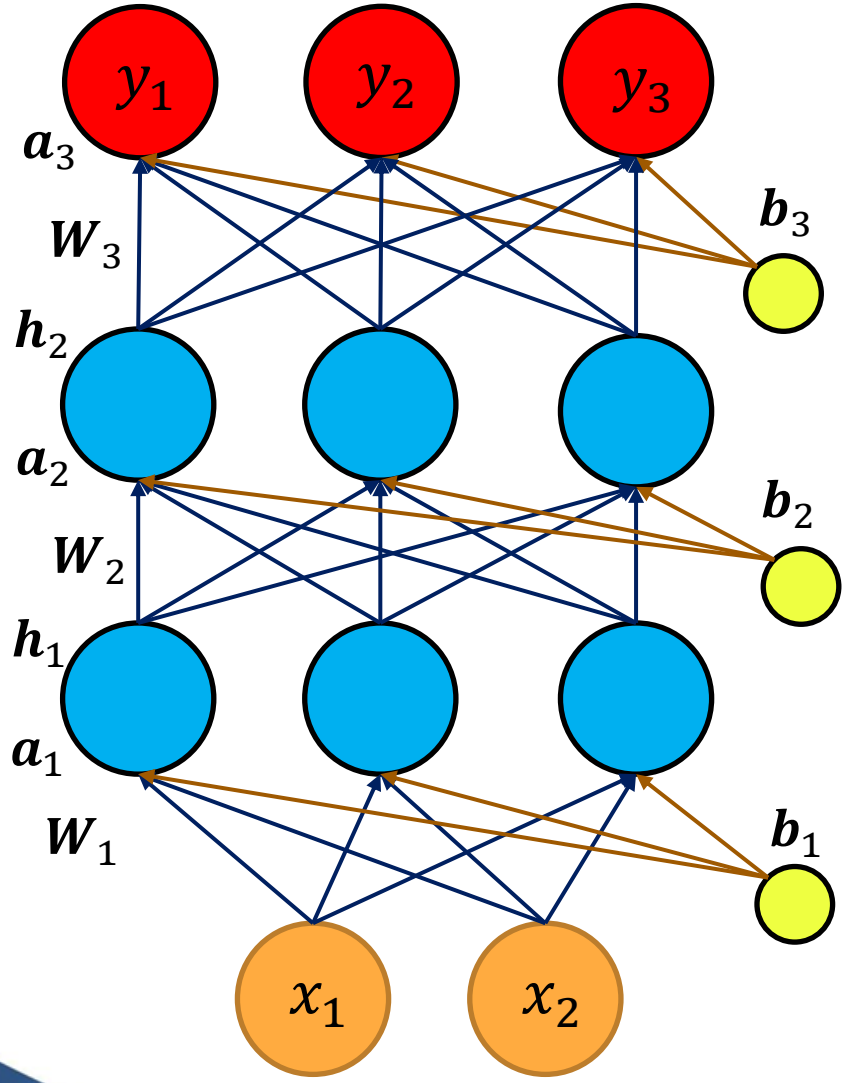
- Inputs: $x_1 = 1.5\ x_2 = 0.5$
- Weights and Biases:

$$\boldsymbol{W}_1 = \begin{bmatrix} 0.53 & 0.86 \\ 1.84 & 0.32 \\ -2.25 & -1.31 \end{bmatrix} \quad \boldsymbol{b}_1 = \begin{bmatrix} -0.43 \\ 0.34 \\ 3.58 \end{bmatrix}$$

$$\boldsymbol{W_2} = \begin{bmatrix} 1.41 & -1.21 & 0.49 \\ 1.42 & 0.72 & 0.03 \\ 0.67 & 1.63 & 0.73 \end{bmatrix} \quad \boldsymbol{b}_2 = \begin{bmatrix} -0.2 \\ -0.12 \\ 1.49 \end{bmatrix}$$

$$\boldsymbol{W}_3 = \begin{bmatrix} -0.3 & 0.89 & -0.81 \\ 0.29 & -1.14 & -2.9 \\ -0.78 & -1.07 & -1.43 \end{bmatrix} \quad \boldsymbol{b}_3 = \begin{bmatrix} 0.32 \\ -0.75 \\ 1.37 \end{bmatrix}$$

Feed Forward Calculation - Example



- Input to hidden layer 1:

$$\boldsymbol{x} = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}$$

- Hidden Layer 1 calculation:

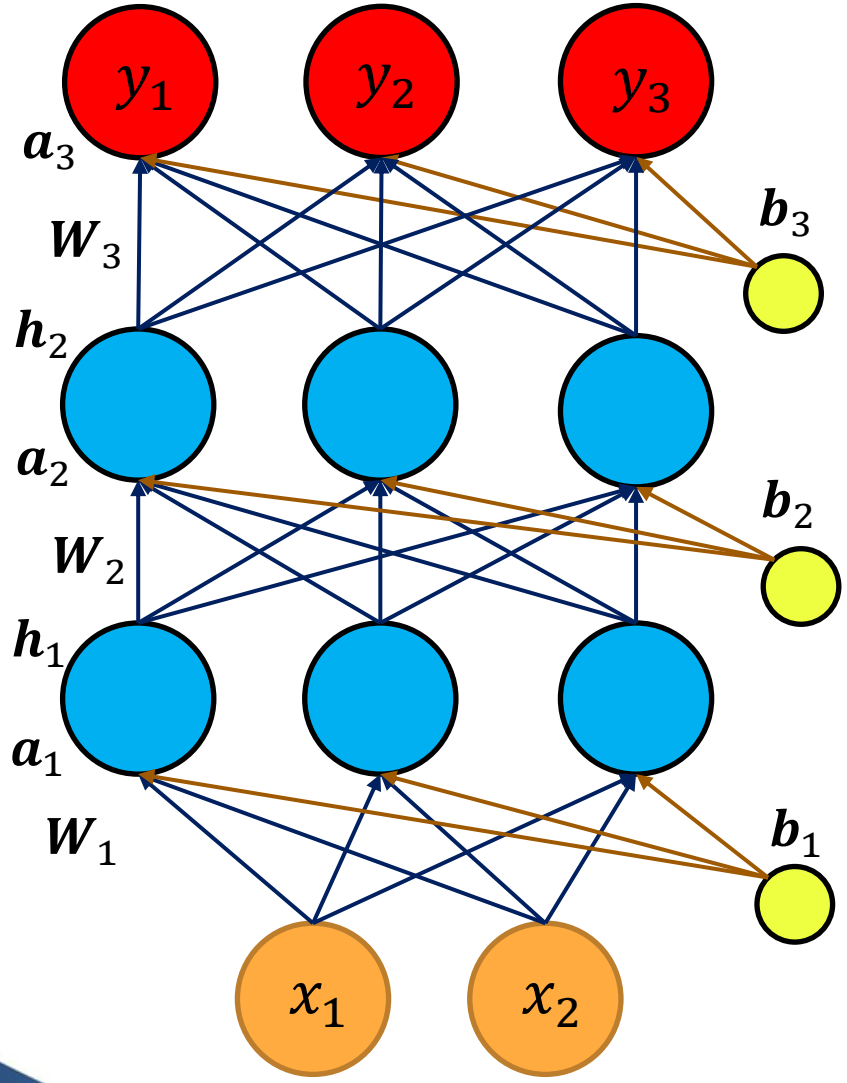
$$\boldsymbol{a}_1 = \boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1$$

$$\boldsymbol{a}_1 = \begin{bmatrix} 0.8 \\ 3.25 \\ -0.46 \end{bmatrix}$$

$$\boldsymbol{h}_1 = relu(\boldsymbol{a}_1)$$

$$\boldsymbol{h}_1 = \begin{bmatrix} 0.8 \\ 3.25 \\ 0 \end{bmatrix}$$

Feed Forward Calculation - Example



- Input to hidden layer 2:

$$\boldsymbol{h}_1 = \begin{bmatrix} 0.8 \\ 3.25 \\ 0 \end{bmatrix}$$

- Hidden layer 2 calculation:

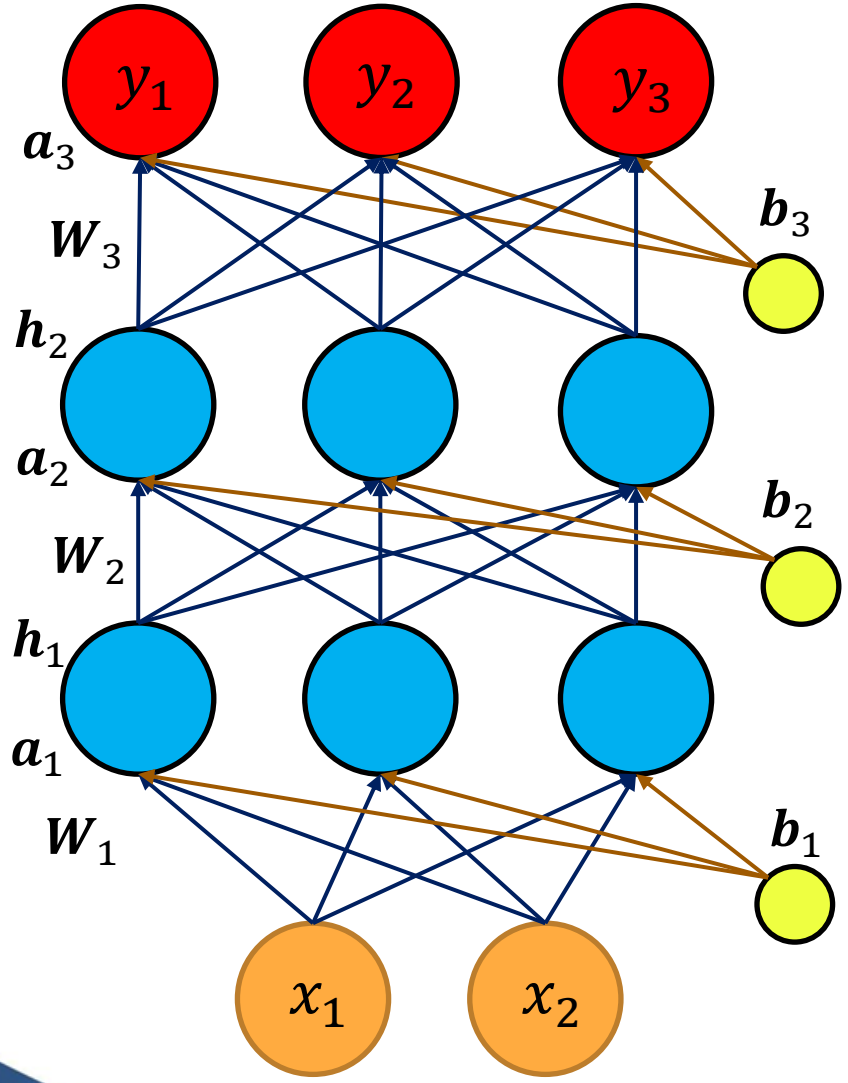
$$\boldsymbol{a}_2 = \boldsymbol{W}_2\boldsymbol{h}_1 + \boldsymbol{b}_2$$

$$\boldsymbol{a}_2 = \begin{bmatrix} -3 \\ 3.34 \\ 7.33 \end{bmatrix}$$

$$\boldsymbol{h}_2 = relu(\boldsymbol{a}_2)$$

$$\boldsymbol{h}_2 = \begin{bmatrix} 0 \\ 3.34 \\ 7.33 \end{bmatrix}$$

Feed Forward Calculation - Example



- Input to output layer:

$$\boldsymbol{h}_2 = \begin{bmatrix} 0 \\ 3.34 \\ 7.33 \end{bmatrix}$$

- Output calculation:

$$\boldsymbol{a}_3 = \boldsymbol{W}_3\boldsymbol{h}_2 + \boldsymbol{b}_3$$

$$\boldsymbol{a}_3 = \begin{bmatrix} -2.63 \\ -26.18 \\ 8.33 \end{bmatrix}$$

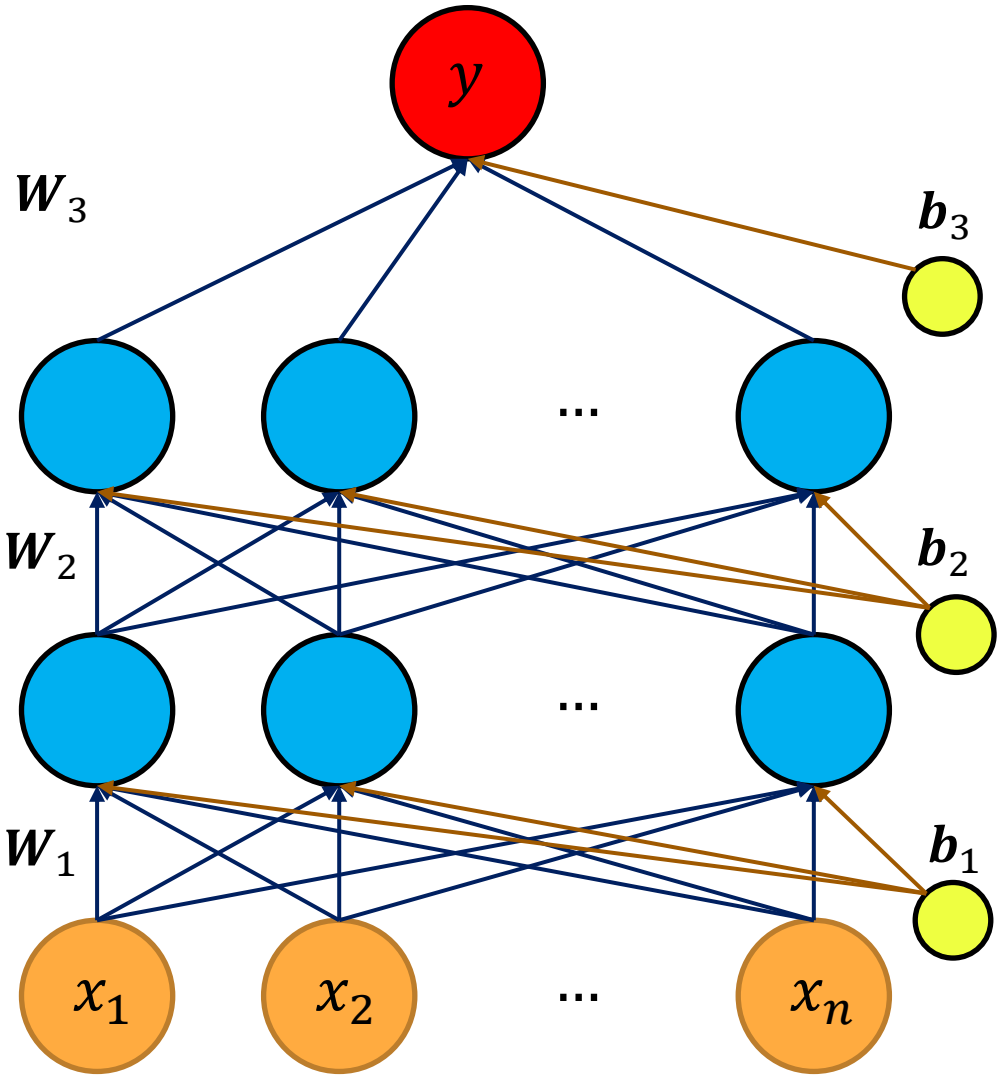
$$\boldsymbol{y} = softmax(\boldsymbol{a}_3)$$

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$ (Approximately)

- This example illustrates how $\boldsymbol{y}$ is a function of $\boldsymbol{x}$ i.e., $\boldsymbol{y} = f(\boldsymbol{x})$

Universal Approximation Theorem

Universal Approximation Theorem (UAT)



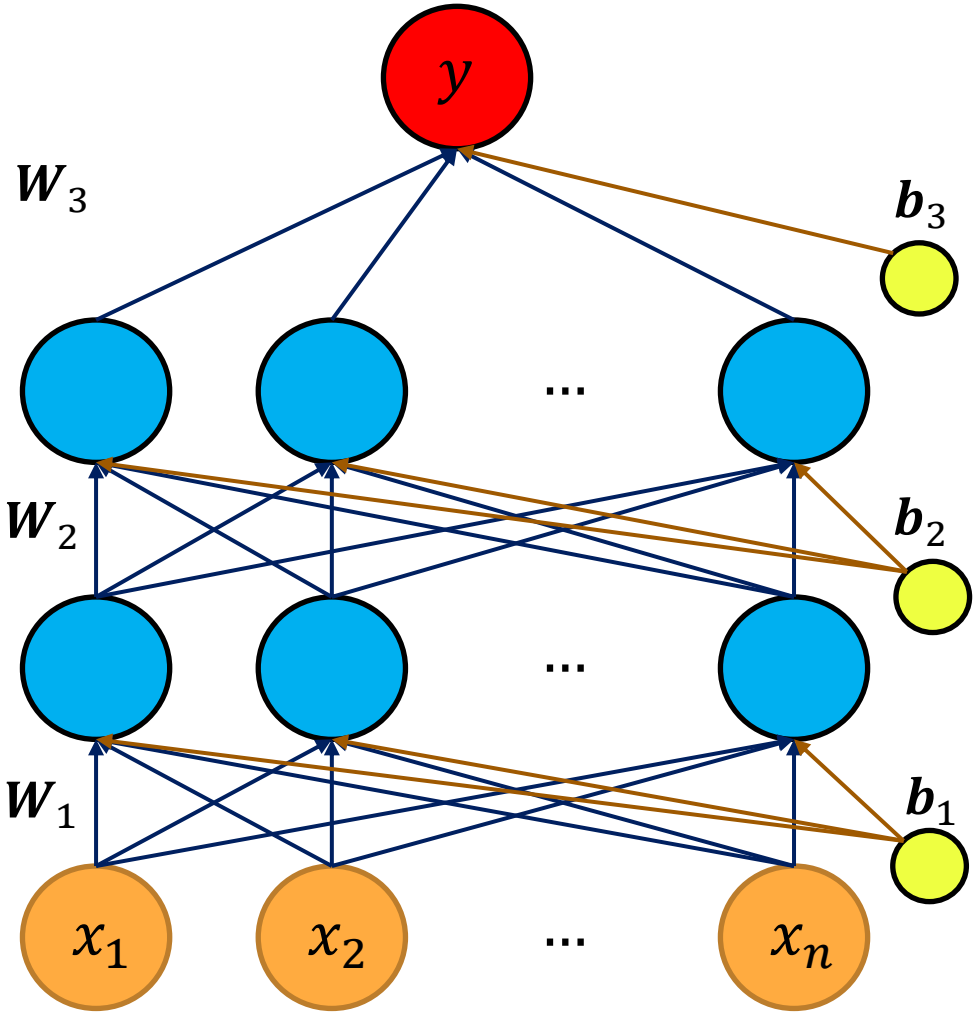
$$y = f(\boldsymbol{x})$$

- Under suitable conditions, UAT establishes that neural networks have a kind of universality in approximating functions
- For any given function of inputs $f(\boldsymbol{x})$, there exists a neural network which can approximate the output
- Condition: Activations functions should be non-linear
- Note: UAT is an existence result; it does not say the network is easy to train or will generalize well

Ref: Article by Micheal Nelson - Neural networks and deep learning

Supervised Learning using NN

Supervised Learning using DNN

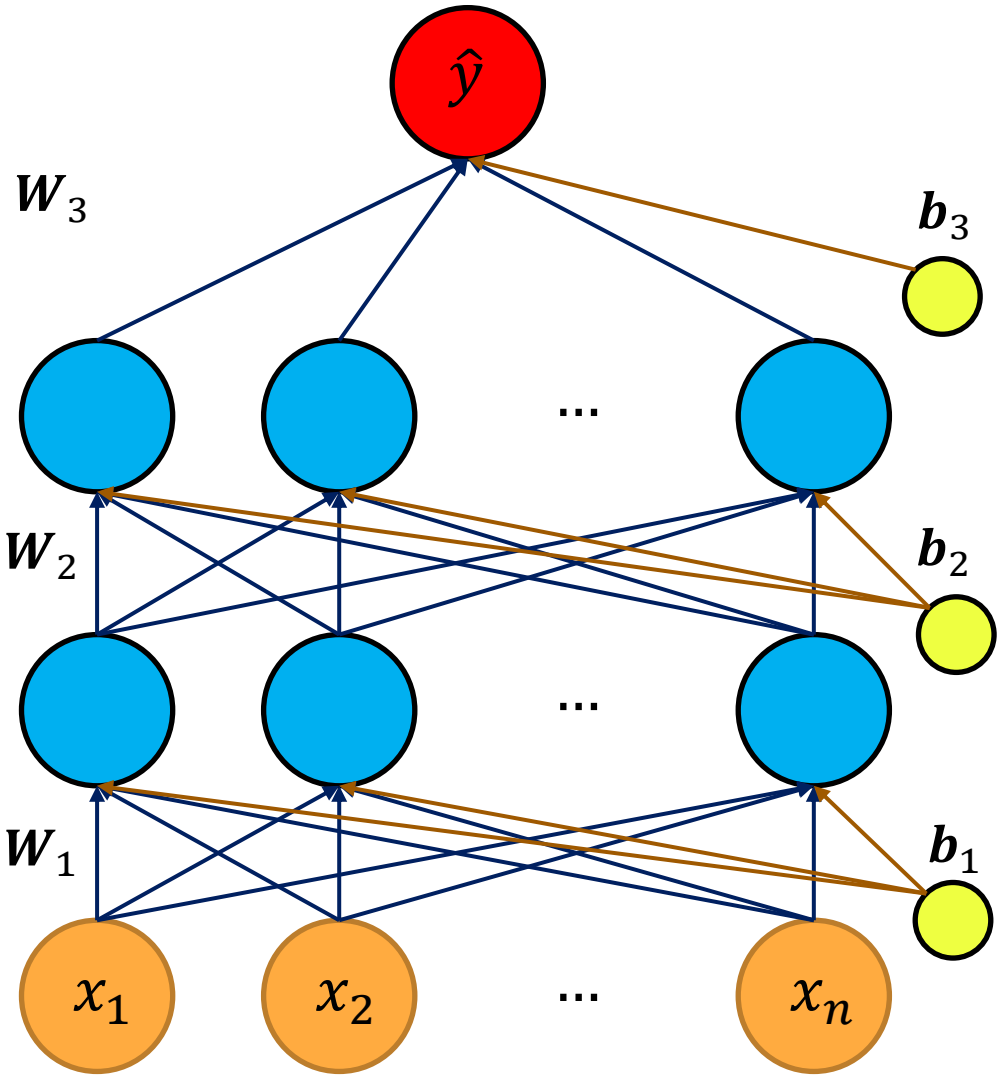


Data for Supervised Learning:

- Inputs: Values of input features – $\boldsymbol{x}$
- Outputs: Values of predicted variables – $\boldsymbol{y}$
  - Regression – Real Numbers
  - Classification – Discrete class or Probability of each class

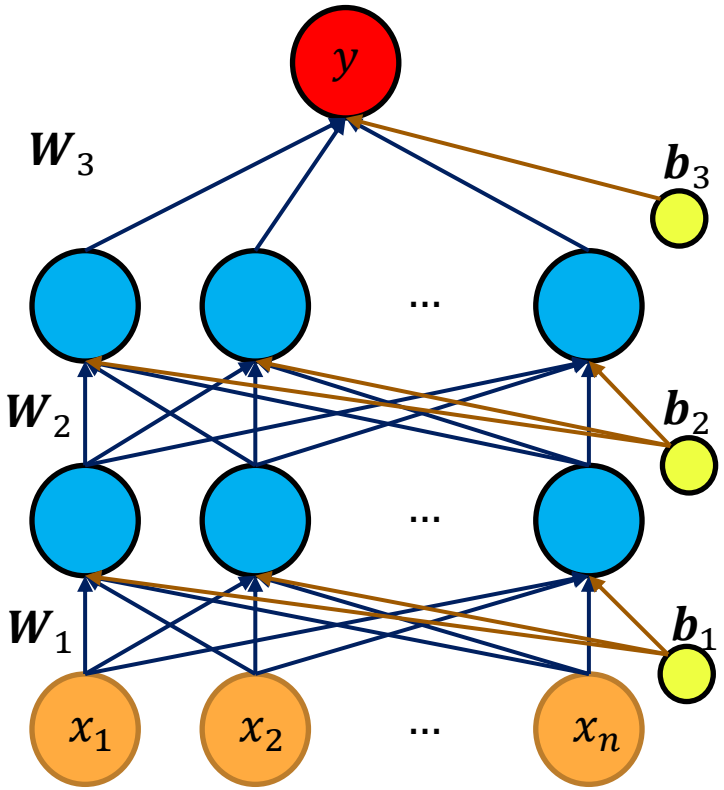
- DNN is expected to take an input and predict the desired output
- Implies: DNN should approximate a function $f(\boldsymbol{x})$ which maps inputs to outputs

Supervised Learning using DNN



- In supervised learning, input $(\boldsymbol{x})$ and output $(\boldsymbol{y})$ data is given to learn a function $\hat{\boldsymbol{y}} = f(\boldsymbol{x})$ such that $\hat{\boldsymbol{y}} \approx \boldsymbol{y}$
- Question: Can we find the weights and biases which will approximate desired $f(\boldsymbol{x})$?
- Training a DNN: Learning the parameters of the DNN (weights and biases) using the given data

Training a DNN



Steps to train a DNN using data

1. Create a neural network
layers
neuron

2. Randomly initialize weights and biases $(W_1, \ldots W_L, b_1, \ldots b_L)$

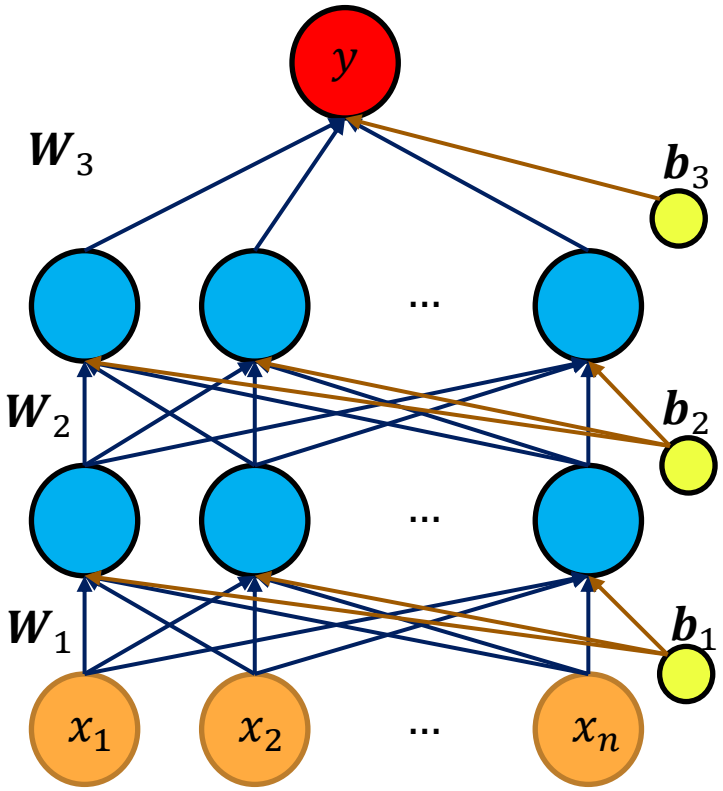
3. Pass inputs (x) through the network and get output $(\hat{y})$

4. Find out difference between actual output (y) and predicted output $(\hat{y})$
- Prediction error

5. Adjust the weights to minimise the difference between y and $\hat{y}$

A DNN based model is trained on the given data
i.e., $\hat{y} \approx y$ for all inputs

Training a DNN – Step 4



Step 4: Find out difference between actual output (y) and predicted output ($\hat{y}$) - Prediction error

- Loss function or Cost function is defined to quantify the prediction error
- For regression: Mean squared error loss since output is a real number

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

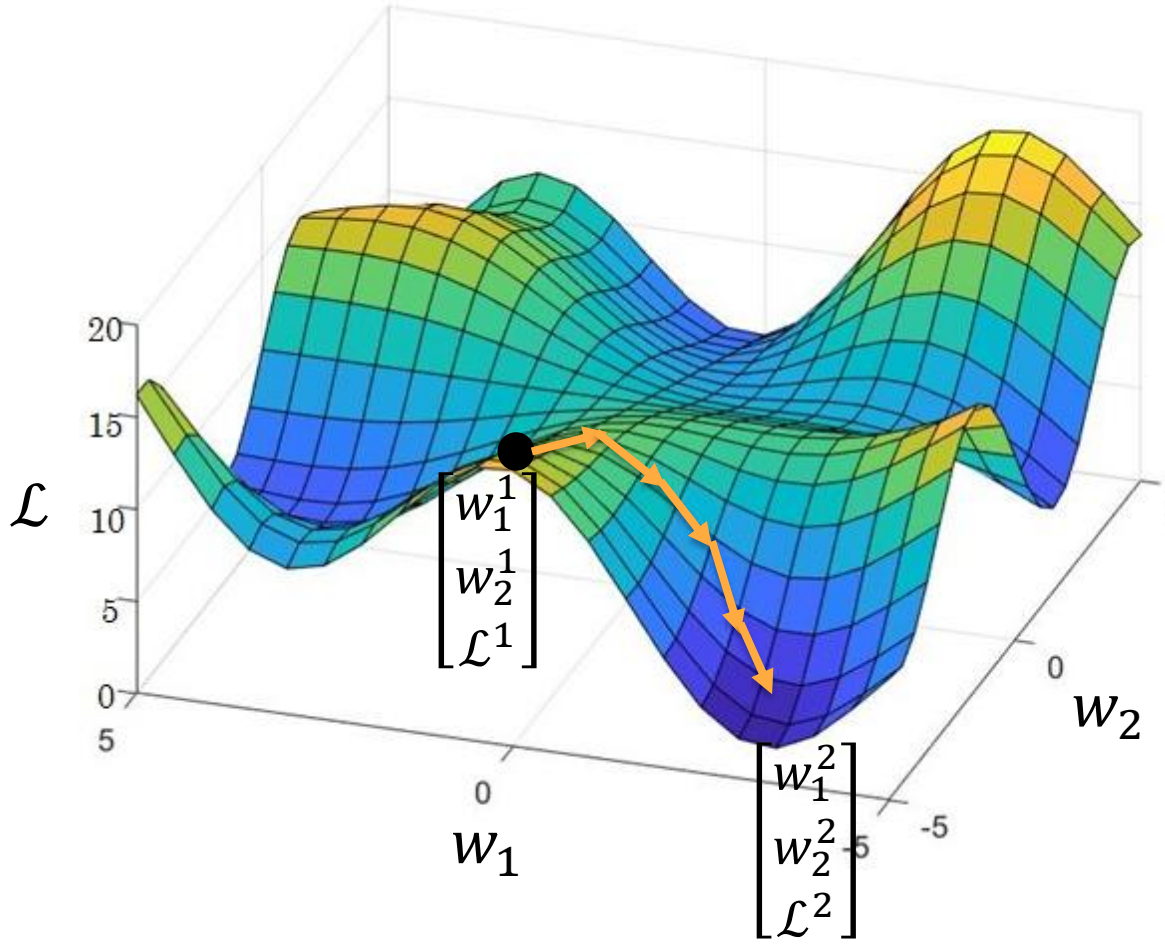
- For classification: Cross entropy loss error since output is a probabilistic value for each class (k classes)

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{1}^{k} y \ln \hat{y}$$

Training a DNN – Step 5

Step 5: Adjust the weights to minimise the loss function

- Question: How to adjust parameters to minimise the loss function $\mathcal{L}(y, \hat{y})$?
- Suppose there are only 2 parameters w_1 and w_2 on which the loss function is dependent
- To minimize loss, take small steps in the direction along which $\mathcal{L}$ decreases
- Implies: w_1 and w_2 are to be modified at each step such that $\mathcal{L}$ decreases
- Directions at each step are given by the gradient



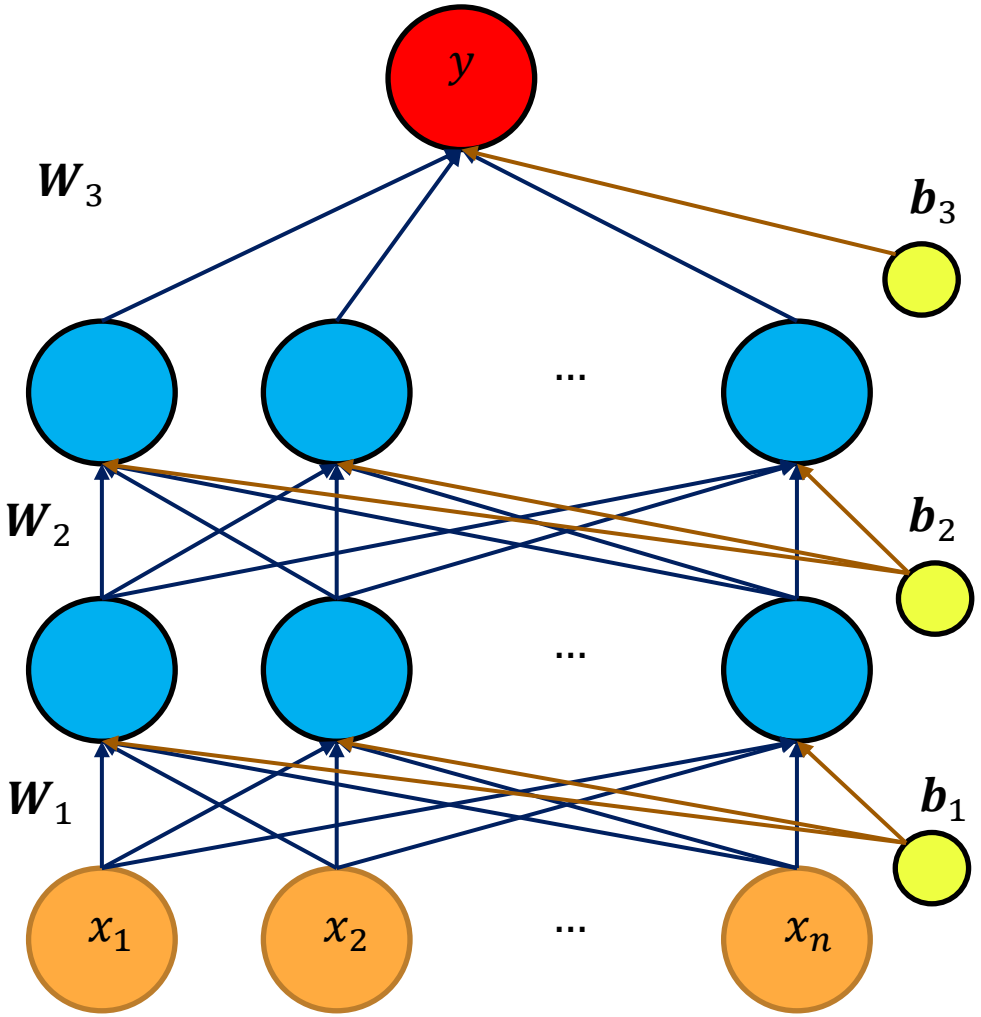
Training a DNN – Gradient Descent

- Note: In a DNN, the loss is dependent on all the parameters $\boldsymbol{W}_1, \boldsymbol{W}_2, \ldots \boldsymbol{W}_L$ and $\boldsymbol{b}_1, \boldsymbol{b}_2, \ldots, \boldsymbol{b}_L$
- So gradient needs to be calculated w.r.t. all the parameters
- General parameter update:

$$w_{new} = w_{old} - \alpha \frac{\partial \mathcal{L}}{\partial w}$$

$$b_{new} = b_{old} - \alpha \frac{\partial \mathcal{L}}{\partial b}$$

- Question: How to calculate the gradient of $\mathcal{L}$ w.r.t. all the parameters in the DNN ?
- Answer: Gradient descent with backpropagation



Gradient Descent with Backpropagation

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \text{ (or)}$$

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{1}^{k} y \ln \hat{y}$$

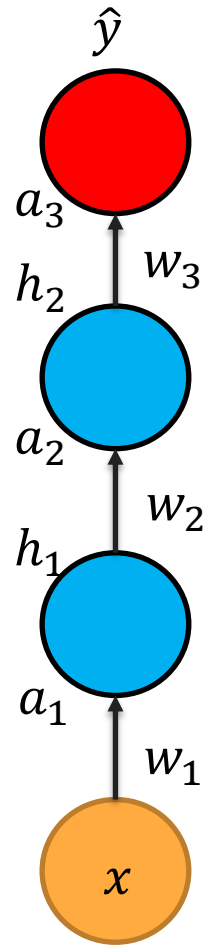
$$\hat{\boldsymbol{y}} = g_o(\boldsymbol{W}_3(\boldsymbol{W}_2(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2) + \boldsymbol{b}_3)$$

- Loss function is connected to the parameters through $\hat{\boldsymbol{y}}$
- Issue: Finding loss function in terms of each of the parameters is a complex process
- Solution: Chain rule can be used to compute the gradient w.r.t each of the parameters
- So the loss is being backpropagated through the chain of gradients

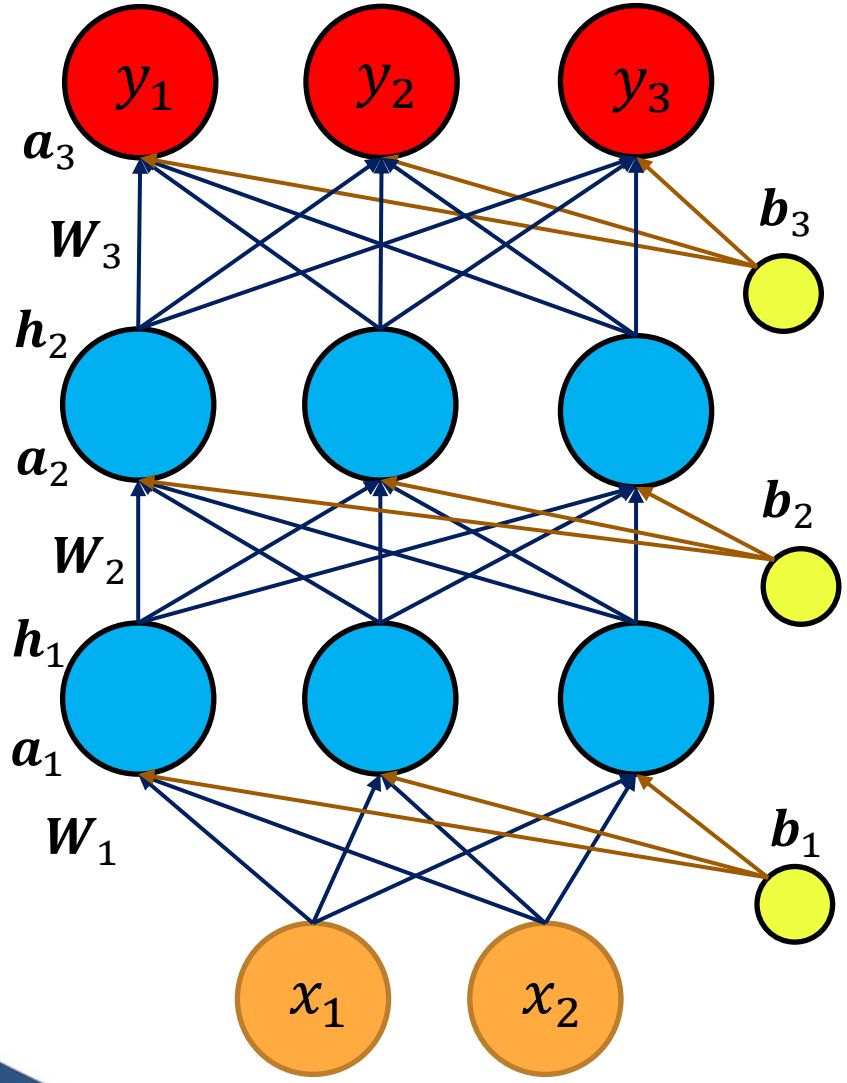
$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial a_3}\frac{\partial a_3}{\partial w_1}$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial a_3}\frac{\partial a_3}{\partial h_2}\frac{\partial h_2}{\partial a_2}\frac{\partial a_2}{\partial h_1}\frac{\partial h_1}{\partial a_1}\frac{\partial a_1}{\partial w_1}$$

Note: $\frac{\partial \mathcal{L}}{\partial w_1}$ will vary for each input and output



Back Propagation Calculation - Example



- Predicted output: $\hat{y} = [0.1 \quad 0.7 \quad 0.2]$
- Target output: $y = [0 \quad 1 \quad 0]$
- Weights and Biases:

$$\boldsymbol{W}_1 = \begin{bmatrix} 0.53 & 0.86 \\ 1.84 & 0.32 \\ -2.25 & -1.31 \end{bmatrix} \quad \boldsymbol{b}_1 = \begin{bmatrix} -0.43 \\ 0.34 \\ 3.58 \end{bmatrix}$$

$$\boldsymbol{W_2} = \begin{bmatrix} 1.41 & -1.21 & 0.49 \\ 1.42 & 0.72 & 0.03 \\ 0.67 & 1.63 & 0.73 \end{bmatrix} \quad \boldsymbol{b}_2 = \begin{bmatrix} -0.2 \\ -0.12 \\ 1.49 \end{bmatrix}$$

$$\boldsymbol{W}_3 = \begin{bmatrix} -0.3 & 0.89 & -0.81 \\ 0.29 & -1.14 & -2.9 \\ -0.78 & -1.07 & -1.43 \end{bmatrix} \quad \boldsymbol{b}_3 = \begin{bmatrix} 0.32 \\ -0.75 \\ 1.37 \end{bmatrix}$$

Training a DNN – Summary of Step 5

a) Known:
   - $(\boldsymbol{x}, \boldsymbol{y}) - N$ Samples
   - $\boldsymbol{W}_1, \boldsymbol{W}_2, \ldots \boldsymbol{W}_L$ and $\boldsymbol{b}_1, \boldsymbol{b}_2, \ldots, \boldsymbol{b}_L$ (initialised values)
   - $\hat{\boldsymbol{y}}$ for each sample and and overall loss $\mathcal{L}$

b) Calculate the gradients of loss function w.r.t. all the weights and biases using chain rule

Note: Gradient calculation involves a summation over all samples in the data

c) Update the weights and biases:

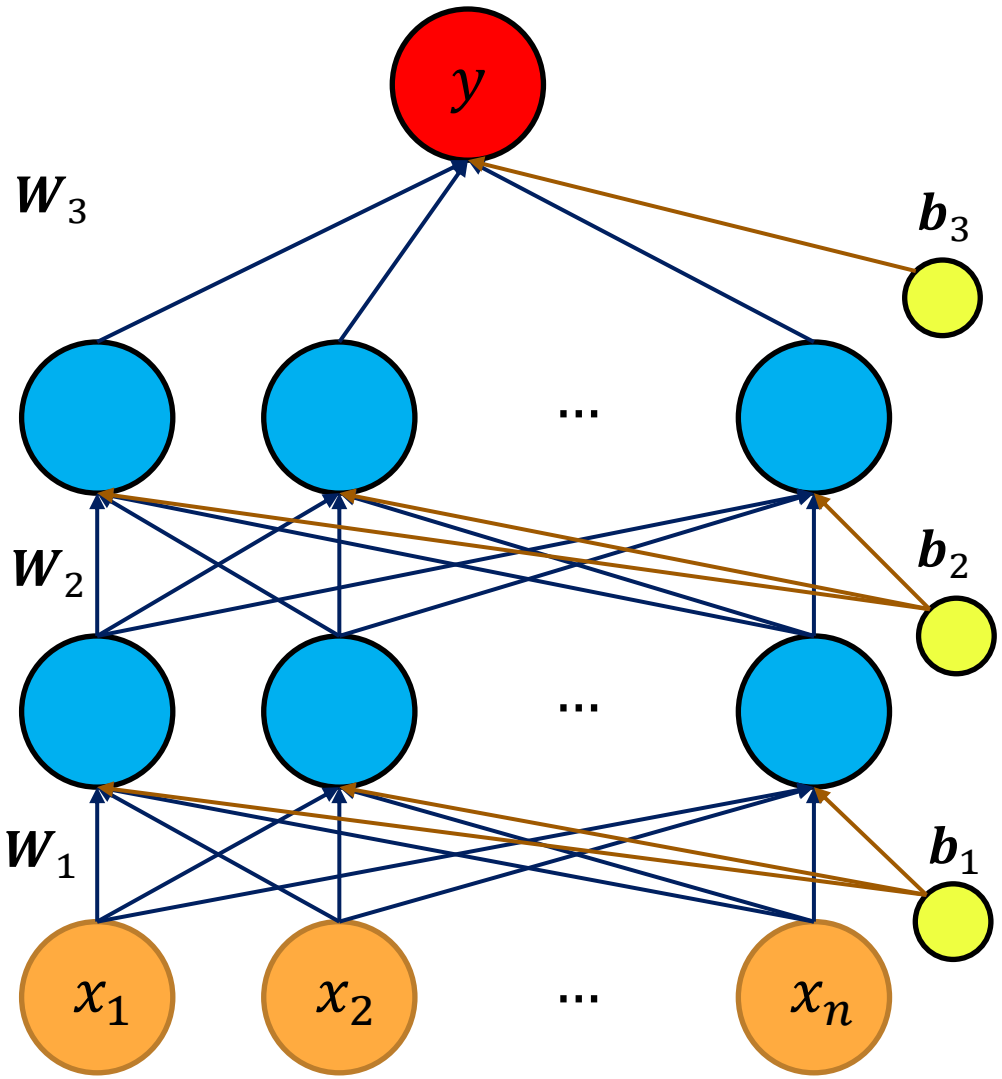
$$w_{new} = w_{old} - \alpha \frac{\partial \mathcal{L}}{\partial w}$$

$$b_{new} = b_{old} - \alpha \frac{\partial \mathcal{L}}{\partial b}$$

d) With new parameter, compute $\hat{\boldsymbol{y}}$ for each sample and and overall loss $\mathcal{L}$

e) Repeat (b), (c) and (d) for many iterations – until loss is minimised

DNN Model Training - Components



- Data (Given): $\{x_i, y_i\}; i = 1 \ldots N$
- Model (Chosen): Regression or Classification

$$\hat{y} = f(x, W_1, \ldots W_L, b_1, \ldots b_L)$$

- Parameters (To be learnt):

$$W_1, \ldots, W_L, b_1, \ldots, b_L$$

- Activation Functions: Relu for hidden layers, linear for regression output layer, softmax for classification output layer
- Loss Function: Mean squared error for regression and cross entropy for classification
- Training Algorithm: Gradient descent with back propagation

Variants of Gradient Descent

Types of Gradient Descent

- Depending on the number of samples used to estimate the gradient of loss w.r.t. the parameters, there are 3 types of gradient descent:
  - Batch Gradient Descent
  - Stochastic Gradient Descent
  - Mini-Batch Gradient Descent

Some Terminology

- Epoch – One epoch of training is said to be complete if every sample in the training dataset is used for gradient calculation and parameter updation
- Batch size (b) – Number of samples used in gradient computation
- Batch size and epoch are the same if all the samples in the dataset are used for computing the gradient

Batch Gradient Descent

- Parameters are updated by estimating the gradient using all the N samples i.e., batch size, $b = N$

$$w_{1(new)} = w_{1(old)} - \alpha\left(\frac{\partial \mathcal{L}}{\partial w_1}\right) = w_{1(old)} - \alpha\left[\left(\frac{\partial \mathcal{L}}{\partial w_1}\right)_1 + \left(\frac{\partial \mathcal{L}}{\partial w_1}\right)_2 + \cdots + \left(\frac{\partial \mathcal{L}}{\partial w_1}\right)_N\right]$$

- In this case, the parameters are updated only once in a epoch
- Advantages: Convergence is guaranteed in this case
- Gradients are estimated in an unbisased manner
- Disadvantage: Slow to converge with large datasets

Stochastic Gradient Descent

- Parameters are updated by estimating the gradient using a single sample
  i.e., batch size, $b = 1$

$$w_{1(new)} = w_{1(old)} - \alpha\left(\frac{\partial \mathcal{L}}{\partial w_1}\right) = w_{1(old)} - \alpha\left[\left(\frac{\partial \mathcal{L}}{\partial w_1}\right)_1\right]$$

$$\vdots$$

$$w_{1(new)} = w_{1(old)} - \alpha\left(\frac{\partial \mathcal{L}}{\partial w_1}\right) = w_{1(old)} - \alpha\left[\left(\frac{\partial \mathcal{L}}{\partial w_1}\right)_N\right]$$

- In this case, the parameters are updated N times in an epoch
- Advantage: Very fast to minimise the loss function
- Disadvantages: Too much variance in gradient calculation and learning might not be stable
- Convergence cannot be guaranteed

Mini-Batch Gradient Descent

- A balance between batch and stochastic gradient descent
- Parameters are updated by using the gradient computed at a small batch of samples i.e., $1 < b < N$

$$w_{1(new)} = w_{1(old)} - \alpha\left(\frac{\partial \mathcal{L}}{\partial w_1}\right) = w_{1(old)} - \alpha\left[\left(\frac{\partial \mathcal{L}}{\partial w_1}\right)_1 + \cdots + \left(\frac{\partial \mathcal{L}}{\partial w_1}\right)_b\right]$$

- In this case, the parameters are updated $\frac{N}{b}$ times in an epoch
- Advantages: Faster than batch gradient descent
- Lesser variance and more stability compared to stochastic gradient descent
- Convergence cannot be guaranteed but most preferred

Variants of Gradient Descent - Visualisation

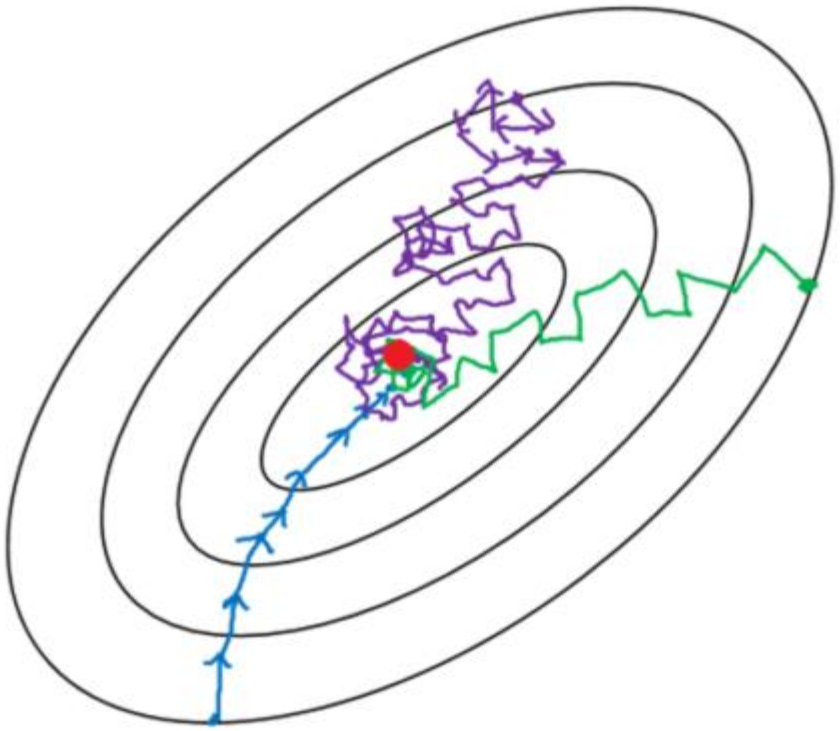


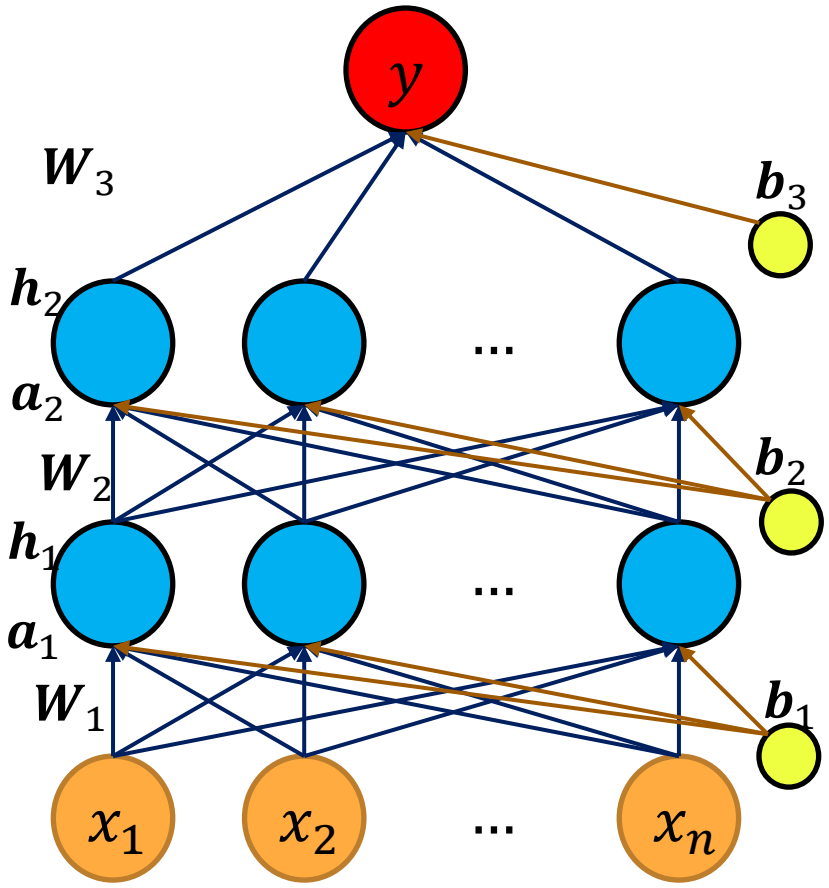
Image Source: medium.com

Batch Gradient descent
Mini-Batch Gradient descent
Stochastic Gradient descent

- Ellipse with higher radius indicates higher loss function

Batch Normalisation

Why Batch Normalisation?



- Problem during training:
  - As weights in earlier layers update, the inputs passed to later layers also change
  - Further, mini-batches contains different set of samples in each batch of training
  - As a result, each layer keeps seeing changing input distributions during training
  - This can make optimization difficult:
    - slower learning
    - unstable gradients
    - greater sensitivity to initialization and learning rate
- Batch normalization helps each layer learn under more stable conditions

Applying Batch Normalisation

- Inputs to each layer are normalised to be unit gaussians before applying the activation function
- Step 1: For each feature in a mini-batch, compute mean (μ) and variance (σ^2)
- Step 2: Normalize: $\hat{h} = \frac{h-\mu}{\sqrt{\sigma^2}}$
- Step 3: Scaling and Shifting with learnable parameters γ and β:

$$y = \gamma\hat{h} + \beta$$

- Step 4: Apply activation function:

$$y_a = g(y)$$

- Benefits of Batch Normalisation:
  - Network learn faster (can use higher learning rates)
  - Leads to faster convergence
  - Acts like a regularizer
  - Reduces sensitivity to initialization
  - Helps prevent exploding/vanishing gradients

Ref: Article by Johann Huber - Batch normalization in 3 levels of understanding

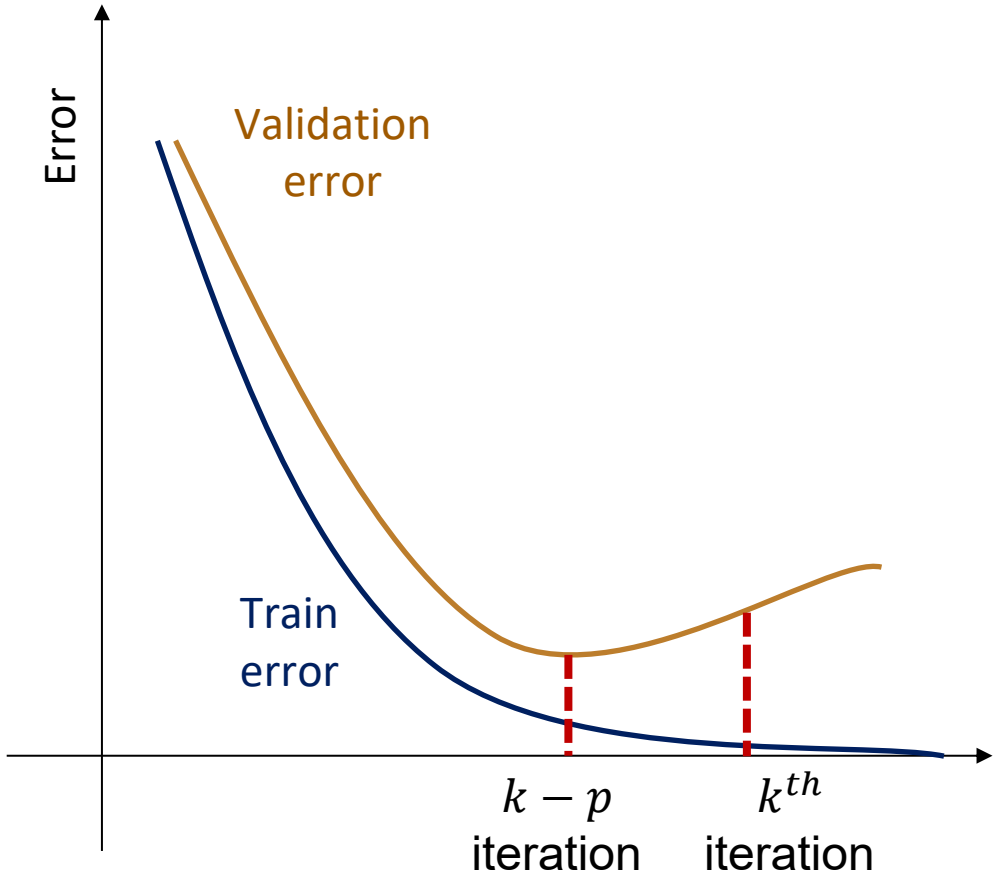
Regularisation

Types of Regularization

- l_1 and l_2 regularization
- Early stopping
- Dropout

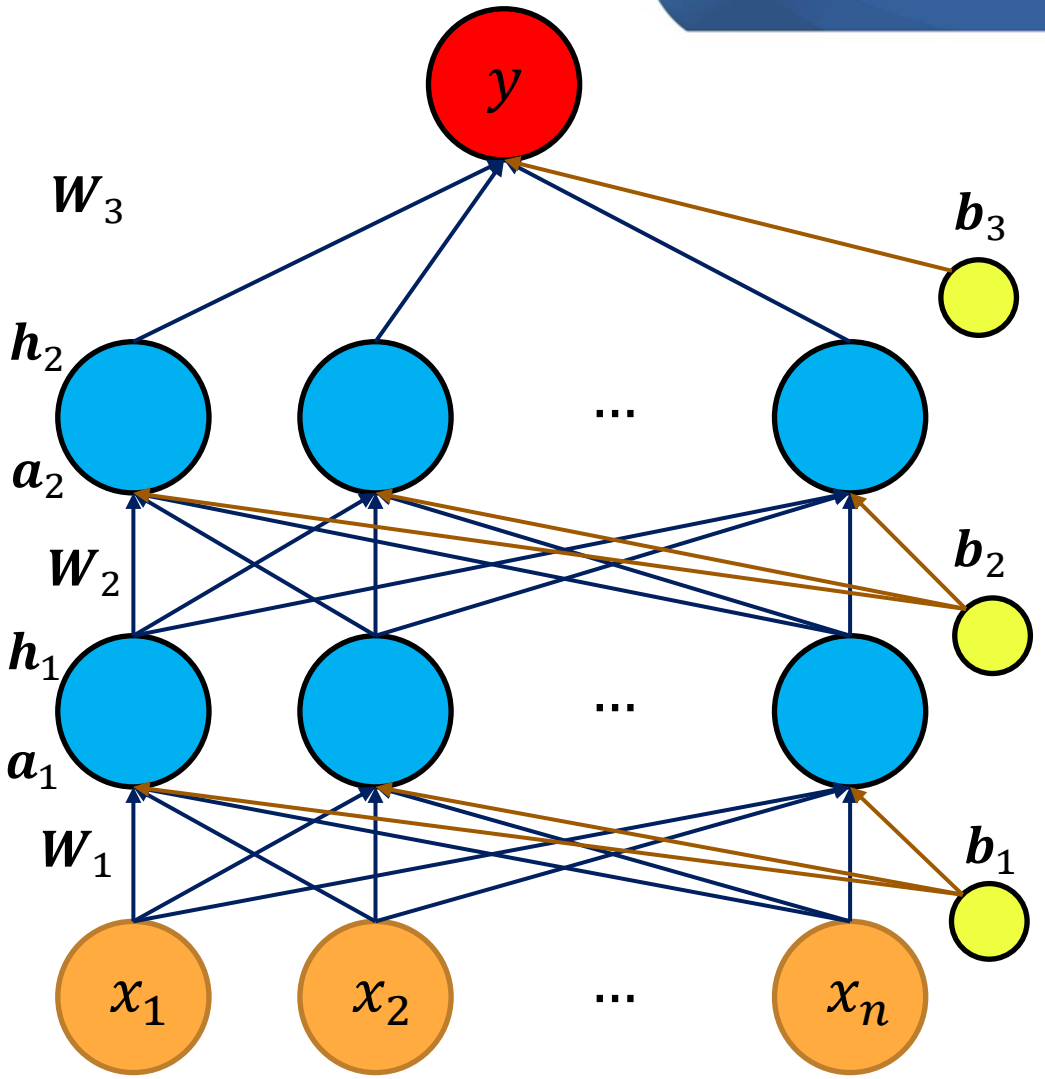
Early Stopping

- Prediction error on a validation set is tracked
- New hyperparameter called patience parameter p is introduced
- Check if there is improvement in the validation error for p continuous iterations
- If not stop and take the model prior to those p iterations



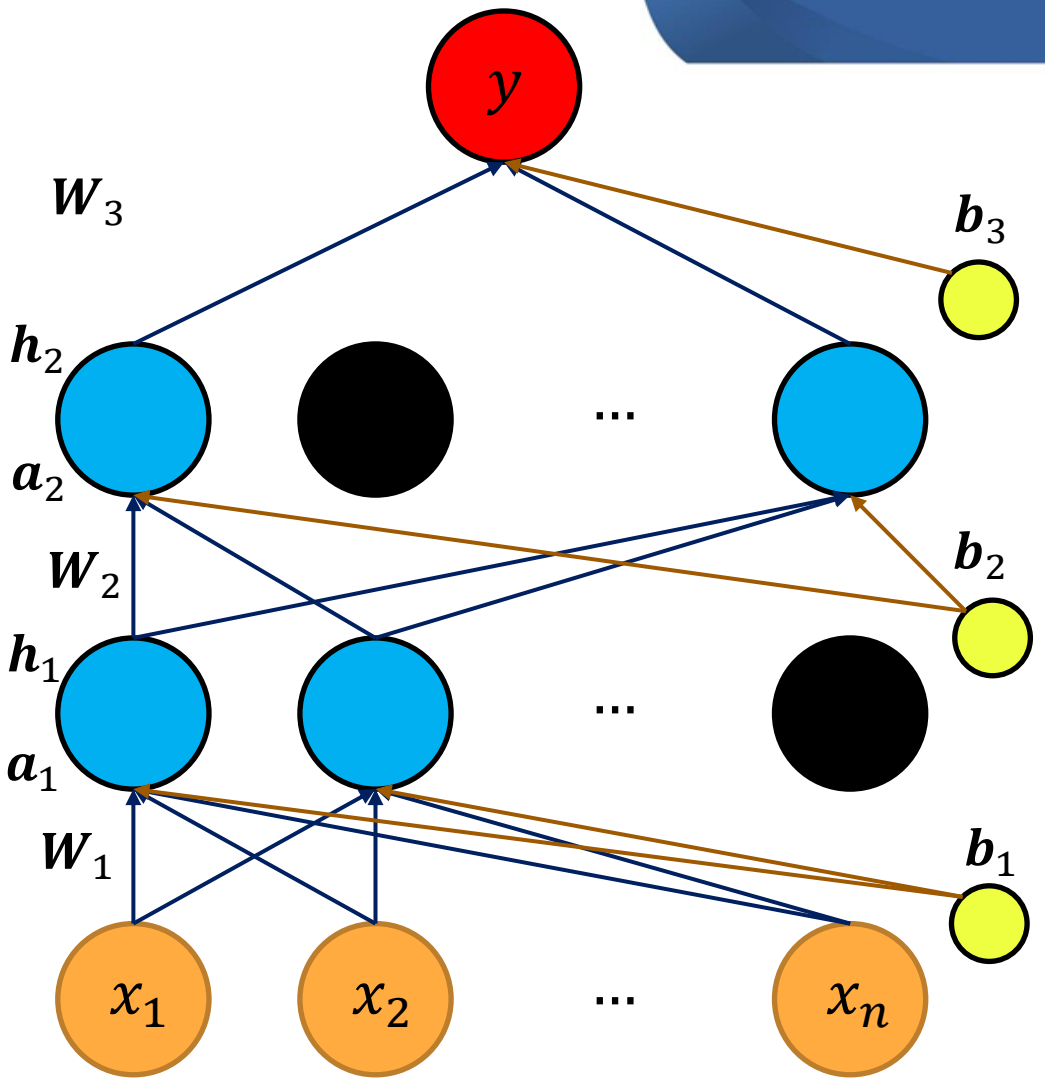
Dropout

- Refers to dropping out of neurons during training
- For an iteration, some neurons with all their connections are removed (inactive)



Dropout

- Refers to dropping out of neurons during training
- For an iteration, some neurons with all their connections are removed (inactive)
- Feed forward calculation and backpropagation happens only with the active connections
- Update the weights and biases of only active connections
- Effectively, learning is happening on a different neural network in each iteration
- Output is equivalent to ensembled output



At i^{th} iteration

DNN Training - Demonstration

- https://playground.tensorflow.org/

Hyperparameter Tuning

Hyperparameters in ANN

- Model Architecture Hyperparameters
  - Number of layers: Determines the depth of the network
  - Number of neurons per layer: Controls the capacity of the model
- Training Hyperparameters
  - Learning rate: Determines the step size for weight updates (crucial for convergence).
  - Batch size: Controls how many samples are processed before updating weights.
- Regularization Hyperparameters
  - L1/L2 regularization factors: Prevents overfitting by penalizing large weights.
  - Dropout rate: Fraction of neurons deactivated to avoid overfitting.

Hyperparameters Tuning in ANN

- Steps to be followed:

Define objective
- Set performance metrics (MSE, accuracy, etc.) and decide to maximise or minimize it

→

Define Search space
- Identify hyperparameters to tune and possible range for each of them

→

Select a Search Strategy
- Select a technique among the ones available for searching hyperparameters

→

Train and Evaluate models
- Train models on training set and evaluate on validation set

↓

Log and Store results
- Keep track of hyperparameters and their corresponding model metrics

→

Select Best Model
- Identify the hyperparameters that yield best performance

→

Check Model for Generality
- Test the model on a separate test set to ensure it generalizes without overfitting

→

Deploy the model
- Deploy the model in production and keep monitoring its performance

Hyperparameters Tuning Techniques

1. Grid Search

- Exhaustively searches all possible hyperparameter combinations.
- Computationally expensive but guarantees finding the best configuration within the search space.

2. Random Search

- Randomly selects hyperparameter values within predefined ranges.
- Often more efficient than grid search in high-dimensional spaces.

3. Bayesian Optimization

- Builds a probabilistic model to predict the best hyperparameters
- Updates the model iteratively based on performance
- Balances exploration (trying new values) and exploitation (refining good values)

Hyperparameters Tuning Techniques

4. Hyperband / Successive Halving

- Dynamically allocates computational resources to evaluate many configurations at start
- Eliminates poor-performing configurations by early stopping
- Efficient for large search spaces by focusing resources on promising candidates.

5. Evolutionary Algorithms / Genetic Algorithms

- Mimics natural selection by evolving hyperparameter populations over generations.
- Starts with a population of random hyperparameter configurations
- Evaluates performance and selects best hyperparameters (Survival of the fittest)
- Combine top hyperparameters to create new configurations (crossover)
- Randomly modify some parameters to introduce diversity (mutation)

Hyperparameters Tuning Techniques

- GridsearchCV from scikit-learn
- RandomisedSearchCV from scikit-learn
- Keras tuner – Specifically handles grid search and random search of hyperparameters in deep learning models
- Auto-Keras – Fully automated model selection and hyperameter tuning using Neural Architecture Search (NAS)
- HyperOpt – Bayesian optimization based search
- NeverGrad – Optimizes hyperparemters using evolutionary algorithms
- Optuna – Support hyperband and early stopping techniques


operation == "MIRROR_X":
irror_mod.use_x = True
irror_mod.use_y = False
irror_mod.use_z = False
operation == "MIRROR_Y":
irror_mod.use_x = False
irror_mod.use_y = True
irror_mod.use_z = False
operation == "MIRROR_Z":
irror_mod.use_x = False
irror_mod.use_y = False
irror_mod.use_z = True
selection at the end -add
ob.select= 1
er_ob.select=1
ntext.scene.objects.acti
("Selected" + str(modifier
irror_ob.select = 0
bpy.context.selected_ob
ata.objects[one.name].se
int("please select exact

THANK YOU